

Gsched: Coordinated Flow-Control and Priority Scheduling for DNN Training in AI Cluster

Heng Xu , Chengze Du , Zhiwei Yu , Letian Li , Ying Zhou , Bo Liu , and Jialong Li , *Member, IEEE*

Abstract—Distributed deep neural network (DNN) training in multi-tenant cloud environments generates long-lived, bandwidth-intensive, and interdependent flows that challenge traditional fairness-based network scheduling. We present Gsched, a framework that orchestrates centralized planning with hardware priority enforcement to realize communication interleaving, thereby eliminating contention and maximizing pipeline efficiency. Unlike methods relying on complex rate control or custom switch hardware, Gsched leverages standard commodity switch features like DSCP and SPQ to enforce strict priority scheduling. It employs a cold-start profiling mechanism to capture task characteristics without user intervention, and a centralized solver to compute the optimal interleaving order that maximizes Average Transmission Efficiency (ATE). Our analysis proves that this prioritization strategy deterministically drives the system to a contention-free state, while a local perturbation theorem guarantees robustness against distributed execution conflicts via network backpressure. Across workloads, Gsched achieves performance close to the state-of-the-art Crux under heterogeneous workloads and remains competitive in highly diverse cases. More importantly, Gsched provides a theoretical foundation that justifies scalable per-bottleneck scheduling in multi-bottleneck networks.

Index Terms—Distributed DNN training, distributed task scheduling, flow control, flow scheduling.

I. INTRODUCTION

LARGE-SCALE distributed deep neural network (DNN) training has become a core workload in modern multi-tenant artificial intelligence (AI) clusters [30], [33], [63], [71]. In traditional cloud environments, to maximize hardware resource utilization, operations typically deploy high-concurrency tasks. However, this strategy is not suitable for DNN training, as its unique traffic patterns and structural characteristics under long-term network saturation make shared network resources a performance bottleneck and pose significant challenges for network scheduling and control. Unlike traffic in conventional cloud applications, which is short-lived, bursty, and independent [38], [46], [58], [76], DNN training traffic is long-lasting,

bandwidth-intensive, and exhibits pronounced cross-flow dependencies [39], [48], [50], [54], which limits the effectiveness of traditional scheduling methods under DNN workloads.

The commonly used synchronous batch processing paradigm alternates between computation and communication phases in each iteration. During the communication phase, operations such as AllReduce for gradient synchronization trigger large-scale, high-bandwidth traffic aggregation across the network. When multiple independent training tasks run concurrently, these periodic traffic bursts cause intense resource contention at network bottlenecks. This raises a fundamental scheduling question: *should the network employ mechanisms such as fair queuing to share bandwidth among concurrent flows, or actively schedule flows to serialize them within the network?*

For DNN workloads with strict synchronization points, the communication overlapping paradigm is suboptimal due to inherent queuing delays and cumulative cross-iteration synchronization effects. In contrast, the communication interleaving paradigm [56], [57] structurally eliminates these performance bottlenecks by orthogonalizing resource allocation in time. However, existing approaches that consider the structured nature of DNN traffic exhibit different trade-offs. Centralized algorithms [3], [23], [53], [77] can optimize performance through global coordination but incur high computational overhead and limited scalability, while distributed algorithms [43], [44], [52], [60] are flexible and easily scalable, but they typically cannot achieve the performance limits attainable by centralized control. Consequently, this study posits a central argument: How to efficiently implement communication interleaving is an important prerequisite for achieving high-performance DNN training.

Once communication interleaving is established as the objective, the next critical challenge is how to efficiently guide the system from a state of uncoordinated concurrent competition to an orderly interleaved state. To address this, this study proposes and analyzes a mechanism based on controllable unfairness: by leveraging strict priority queuing to enforce exclusive bandwidth occupancy for high-priority tasks, the entire multi-task system can be deterministically driven to converge exponentially toward the ideal interleaved state. The key advantage of this approach is that it does not require any custom hardware modifications and instead utilizes standard commodity switch features as a readily deployable mechanism, enabling efficient collaborative network scheduling in distributed environments.

However, establishing interleaving only achieves the serialization of flows and does not directly determine system performance. In fact, different service orders of the serialized flows

Received 24 January 2026; revised 5 May 2026; accepted 17 May 2026. Date of publication 20 May 2026; date of current version 3 June 2026. This work was supported by the Guangdong Provincial Young Innovative Talent Program under Grant 2025KQNCX077. Recommended for acceptance by Dr. Xiang Cheng. (Corresponding author: Jialong Li.)

Heng Xu, Chengze Du, Ying Zhou, Bo Liu, and Jialong Li are with the Faculty of Computer Science, Artificial Intelligence, Shenzhen University of Advanced Technology, Shenzhen 518107, China (e-mail: lijialong@suat-sz.edu.cn).

Zhiwei Yu is with the Institute for Network Sciences, Cyberspace, Tsinghua University, Beijing 100084, China.

Letian Li is with the Department of Information Engineering, Chinese University of Hong Kong, Shatin Hong Kong.

Digital Object Identifier 10.1109/TNSE.2026.3695132

correspond to different optimization objectives. To address this, this study proposes a general scheduling method, which is based on the serialized flow sequence and constructs the scheduling sequence through adjacent flow swapping. Using this method, we maximize the Average Transmission Efficiency (ATE) by systematically adjusting the flow order to optimize overall transmission performance.

Translating this comprehensive scheduling theory into a practical and scalable distributed system poses significant engineering challenges. We propose a local perturbation theorem. It shows that the optimal sequence has strong structural stability. Local adjustments caused by distributed asynchronous decisions have minimal impact on global performance. This provides a theoretical basis for constructing a robust distributed system. Additionally, the perfect interleaving feasibility condition defines the deployment boundaries and performance limits of the framework.

To realize this theory in practice, we propose Gemini Scheduling (Gsched), which coordinates centralized planning with distributed priority enforcement. While the state-of-the-art Crux achieves excellent performance through GPU-intensity heuristics and DSCP-based priorities, Gsched provides a complementary theoretical foundation (Theorem 4) for decomposing global contention into lightweight local decisions with limited performance loss. Gsched profiles task characteristics, computes the interleaving order, and maps decisions to hardware priority queues. Together with flow control, this design gives critical flows exclusive bandwidth, resolves distributed conflicts, and improves task progress and resource utilization.

The performance of Gsched is evaluated through experiments in our cluster test environment, together with trace studies derived from real distributed training runs. The results show that Gsched performs close to the state-of-the-art Crux in common homogeneous and semi-homogeneous settings, while remaining competitive across heterogeneous workloads. In highly diverse cases, Gsched can better exploit ATE-based ordering to reduce communication stalls and improve bottleneck utilization. These results suggest that Gsched is not intended to replace strong heuristic schedulers such as Crux, but to provide a complementary scheduling framework with a theoretical basis for per-bottleneck decomposition.

II. CHALLENGES

This section identifies the key scheduling challenges in multi-tenant DNN training and motivates coordinated interleaving with flow control.

Incompatibility of Traditional Objectives: DNN training traffic (Fig. 1(b)) exhibits long-lived, persistent flows with strict computation-communication alternation and AllReduce synchronization, unlike the short-lived, independent flows in traditional datacenters (Fig. 1(a)). Traditional fairness or average FCT objectives fail because iteration time is bottlenecked by the slowest node: any delay in critical flows stalls the entire task. Diverse parallelism patterns (data, model, pipeline) further complicate unified optimization, as contention and inter-flow dependencies are ignored by conventional metrics.

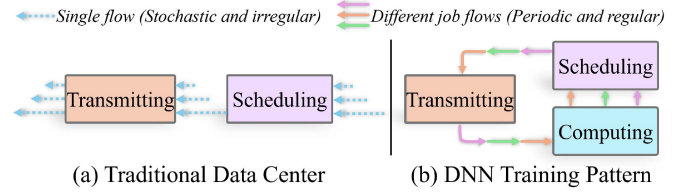


Fig. 1. Traffic pattern in (a) traditional datacenters and (b) DNN training clusters.

Limitations of Existing Architectures: Centralized scheduling incurs high overhead from global state collection and struggles with heterogeneous, dynamic DNN workloads, causing decision latency that cannot match rapid training phases. Distributed scheduling treats flows independently, overlooking task-level interdependencies and leading to globally suboptimal outcomes. Flow control lacks application semantics, responds slowly to congestion, and cannot prioritize critical synchronization flows, resulting in delayed convergence and poor adaptability.

The Case for Coordinated Interleaving: Communication interleaving exploits asynchrony across tenants to fill communication gaps and maximize link utilization without delaying critical paths. It redefines fairness from flow-level bandwidth sharing to task-level progress. However, under congestion, independent scheduling and flow control often work against each other. True optimization therefore requires co-design, where scheduling reacts to network feedback and flow control enforces application priorities. This observation motivates our controllable-unfairness mechanism and the ATE metric for driving the system toward contention-free interleaved states on commodity switches.

III. RELATED WORK

This section reviews work related to Gsched across transport protocols and queue management (Section III-A), flow scheduling and load balancing (Section III-B), cluster scheduling (Section III-C), and communication-aware DNN optimization (Section III-D). Table I summarizes their key differences.

A. Evolution of Transport Protocols

The fundamental layer of data center networking relies on robust congestion control and transport protocols, which have evolved significantly to handle diverse traffic patterns.

Early approaches primarily utilized packet loss as the congestion signal. Protocols such as TCP Reno [31], BIC [73], and its successor CUBIC [24] aggressively probe bandwidth but often fill switch buffers, leading to high latency. TCP Westwood [22] attempted to improve performance over wireless or lossy links by estimating bandwidth, yet it shares the inherent limitations of loss-based control in high-speed datacenters.

To mitigate bufferbloat, delay-based algorithms were introduced. TCP Vegas [11] pioneered the use of RTT variations to detect congestion before packet loss occurs. Modern iterations like Swift [36] and Yeah-TCP [10] further refine this concept,

TABLE I
COMPARISON OF GSCHED WITH DIFFERENT FLOW SCHEDULING AND CONTROL MECHANISMS

Category & Methods	Scheduling Scope	Primary Objective	DNN Awareness	Comm. Strategy	Hardware Requirement
<i>Transport Protocols</i>					
DCTCP [6]	Packet-level	Fairness & Low Latency	No	Fair Sharing	Commodity (ECN)
DCQCN [81]	Packet-level	Fairness & Low Latency	No	Fair Sharing	Commodity (ECN)
Homa [49]	Packet-level	Tail Latency	No	SRPT	Commodity
NDP [25]	Packet-level	Incast & Tail Latency	No	Packet Trimming	Commodity
<i>Flow Scheduling</i>					
pFabric [4]	Packet/Flow	Minimize FCT	No	SRPT	Commodity
Coflow [15]	Flow Collection	Minimize CCT	No	Rate Control	Commodity
MLTCP [57]	End-host Flow	Approx. Interleaving	Yes	Rate Control	Commodity
<i>Cluster Scheduling</i>					
Gandiva [70]	Job-level	Cluster Efficiency	Yes (Compute)	N/A	Commodity
Pollux [54]	Job-level	Adaptive Goodput	Yes (Compute)	N/A	Commodity
<i>DNN Comm. Optimization</i>					
ByteScheduler [52]	Intra-job	Hide Comm. Latency	Yes	Overlapping	Commodity
TicTac [26]	Intra-job	Hide Comm. Latency	Yes	Overlapping	Commodity
CASSINI [56]	Inter-job	Collision Avoidance	Yes	Staggering	Commodity
Crux [13]	Inter-job	GPU Utilization	Yes	Priority	Commodity
Gsched (Ours)	Inter-job	Max Link Utilization	Yes	Perturbation stable ordering	Commodity

utilizing precise timestamping to maintain low target delays while sustaining high throughput.

However, in modern datacenters, Explicit Congestion Notification (ECN) has become the standard for conveying network states. DCTCP [6] leverages ECN marks to provide fine-grained, proportional rate reduction, while DCQCN [81] extends this to RDMA over Converged Ethernet (RoCEv2), becoming a staple for high-performance AI clusters.

MQ-ECN [8] enables ECN in multi-service multi-queue datacenters by separating marking behavior across service classes. ACC [74] automatically tunes ECN marking thresholds in high-speed datacenter networks to maintain short queues under changing traffic. ABM [1] actively manages shared switch buffers by considering both total buffer occupancy and queue drain time, improving burst absorption and queue isolation. MicroBurst-ECN [75] improves ECN marking for micro-burst traffic in multi-queue datacenters by decoupling threshold setting and marking behavior. MPCC [79] applies model predictive control to datacenter congestion control and adjusts congestion windows according to queue dynamics.

Beyond these standard paradigms, specialized protocols have emerged to address specific constraints. Deadline-aware protocols like D2TCP [68] incorporate flow deadlines into the congestion window calculation to meet SLA requirements. In-network congestion control schemes, such as XCP [35], RCP [21], and RoCC [64], utilize active feedback from switches to explicitly dictate sending rates, offering faster convergence than end-host adaptation. Furthermore, receiver-driven designs like NDP [25] and Homa [49] fundamentally shift flow control responsibility to the receiver, using packet trimming and priority mechanisms to handle incast patterns and micro-bursts effectively.

TIMELY [47] uses RTT as a delay signal for datacenter congestion control, HPCC [40] leverages in-band telemetry to obtain precise link-load information, PowerTCP [2] adapts the sending behavior to high-speed bandwidth-window dynamics, and Bolt [7] targets sub-RTT congestion control for ultra-low latency in fast datacenter networks. eTran [14] enables extensible

kernel transport with eBPF for customized congestion control and packet scheduling.

While these mechanisms optimize packet-level metrics and enforce flow fairness, they operate agnostically to the upper-layer application logic.

B. Flow-Level and Coflow Scheduling

Moving up the stack, flow scheduling algorithms aim to optimize FCT rather than just link fairness.

pFabric [4] demonstrates near-optimal FCT by prioritizing packets based on the shortest remaining flow size, though it requires custom switch hardware modifications.

Recognizing that applications often generate collections of flows with shared goals, the concept of Coflow was introduced. The initial Coflow architecture [16] and subsequent optimizations like Varys [15] and Aalo [32] coordinate the scheduling of flow groups to minimize the completion time of the slowest flow in the group. These methods are particularly effective for MapReduce-style data-parallel stages.

A separate line of work studies traffic scheduling and load balancing in datacenter networks. PIAS [9] approximates shortest-job-first scheduling with commodity priority queues when flow sizes are unknown, PDQ [29] performs preemptive flow scheduling to reduce flow completion time and deadline misses, and Baraat [18] schedules flows at the task level to reduce head-of-line blocking. CONGA [5] uses congestion-aware flowlet assignment, Presto [28] performs edge-based load balancing with flowcells, LetFlow [69] exploits flowlet elasticity for asymmetric paths, and ConWeave [62] supports fine-grained RDMA rerouting with in-network reordering.

Despite their effectiveness in general data analytics, these methods fall short for deep learning.

C. Centralized Network and Cluster Scheduling

Scheduling in datacenters is not limited to flows; it also encompasses network-wide traffic engineering and compute resource management.

Centralized network schedulers like Hedera [3] and FastPass [53] utilize a global controller to orchestrate routing and transmission times, achieving near-optimal network utilization. However, the latency of the central controller limits their scalability and responsiveness to microsecond-scale bursts.

The unique challenges of DNN training traffic, specifically its periodic, bursty, and synchronized nature, have been extensively characterized in works such as Echelon [50], MLNet [44], and Comet [38]. These studies highlight that traditional resource scheduling is insufficient for AI workloads.

Consequently, a rich body of work has focused on GPU cluster scheduling. Early systems like Gandiva [70], Themis [42], and Tiresias [23] focused on maximizing cluster efficiency through time-slicing, placement locality, and minimizing JCT.

Later systems introduced greater adaptivity: Pollux [54] and Optimus [51] dynamically adjust batch sizes and resources based on training progress. Shockwave [78] utilizes market-based mechanisms for fair scheduling. SIA [33] explicitly addresses hardware heterogeneity. TORTA [19] extends optimization to distributed inference using reinforcement learning.

While these schedulers excel at GPU packing and job placement, they typically treat the network as a static bottleneck or a secondary constraint. They do not actively manage the real-time interleaving of network flows, leaving communication contention to be handled by the underlying, often inefficient, TCP/RDMA logic.

D. Communication-Aware DNN Optimization

Recent research has increasingly focused on co-optimizing communication alongside computation, divided into intra-job and inter-job strategies.

To accelerate individual training tasks, systems optimize the execution of collective primitives. TACCL [60] synthesizes topology-aware communication algorithms, while BytePS [34] utilizes unified bandwidth management for parameter servers. Pipelining techniques, such as TicTac [26], and fine-grained tensor schedulers like ByteScheduler [52] and Syndicate [43], break down large tensors to overlap communication with computation, effectively hiding network latency within the training loop. US-Byte [65] optimizes the merging and ordering of unequal-sized tensor blocks, while D-Credit [66] dynamically adjusts the simultaneous communication window for tensor blocks. EchelonFlow [67] further models flow completion under specific distributed training paradigms and aligns communication with computation through ideal completion times.

Mercury [20] moves DNN communication scheduling to the transport layer by assigning packet-level priorities. Rails [72] addresses sparse and imbalanced All-to-All traffic in expert-parallel MoE training through topology-aware load balancing. FAST [37] schedules All-to-All transfers by keeping the inter-server bottleneck highly utilized and using fast intra-server networks to mitigate stragglers. UCCL [80] provides an extensible software transport layer for GPU networking by decoupling RDMA NIC data and control paths, enabling multipath transport for ML collectives.

Addressing the contention between multiple concurrent jobs is the most relevant frontier to our work. Muri [77] introduces

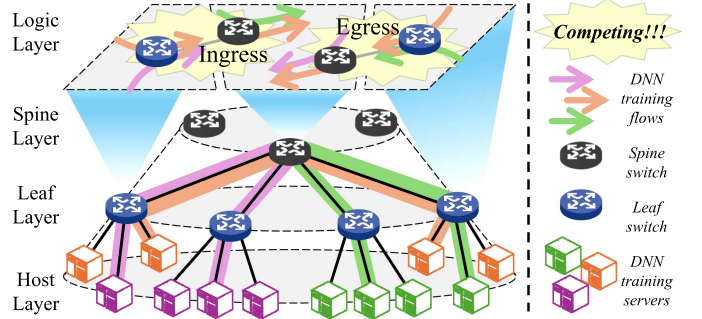


Fig. 2. Spine-Leaf network topology.

multi-resource interleaving to optimize GPU sharing across jobs. MLTCP [57] shows the benefit of controlled unfairness for staggering communication across concurrent jobs, but it does not provide a centralized scheduling framework with hardware-enforced priority execution. CASSINI [56] uses geometric models to schedule job phases, aiming to desynchronize their peak communication demands. Crux [13] represents a state-of-the-art approach that prioritizes flows based on GPU intensity to mitigate interference.

While these approaches, including Crux's strong heuristic, make significant progress through practical workload proxies such as GPU intensity, Gsched takes a complementary direction. It studies how a system-centric ordering objective can be decomposed into per-bottleneck decisions with bounded loss under local perturbations. The local perturbation stability theorem shows that conflict-induced priority swaps can be interpreted as adjacent exchanges with limited degradation. This theoretical justification, rather than packet marking itself, is our main conceptual contribution.

IV. OVERVIEW

This section introduces Gsched's operating assumptions (Section IV-A), scheduling objective (Section IV-B), theoretical roadmap (Section IV-C), and system design (Section IV-D).

A. Basic Concepts and Operating Environment

Multi-tenant Environment: Gsched targets shared GPU clusters where multiple independent DNN training tasks compete for limited network bandwidth. Workload heterogeneity and resource contention make efficient arbitration a central challenge.

DNN Training Traffic Pattern: Different parallelism strategies generate distinct traffic patterns. Here, we simply use data-parallel training as an illustrative example, where traffic follows a loop of computation, transmission, aggregation, transmission, and synchronization before the next computation. Within a single iteration, computation and communication alternate multiple times, with varying durations reflecting the heterogeneity of forward and backward passes and communication steps.

Network Model: In Fig. 2, the logical layer of DNN training network topology in competition is actually composed of multiple bottleneck links. In complex data center networks, a flow's path may traverse several bottlenecks such as switch egress ports.

The model accounts for flows relating to multiple bottlenecks and flows competing at various bottleneck points.

System Operating Conditions: Our study assumes abundant computation resources of GPUs and focuses on network resources as the bottleneck. With many concurrent training tasks deployed to maximize costly cluster utilization, network resources become saturated over time. This work addresses such network-induced bottlenecks by improving overall performance through efficient scheduling.

Deployable Load and Placement: We assume a standard resource scheduler performs admission control based on computation and communication demands, so the aggregate bandwidth demand of tasks sharing a bottleneck does not exceed capacity C . Under this admissible load, strict priority queuing acts as serialization rather than starvation. High-priority queues are served first, while lower-priority tasks remain serviceable within the iteration window because the switch is work-conserving and total demand is bounded.

Scope and Applicability: Gsched is designed for communication-critical multi-tenant DNN training scenarios in which network contention is the primary limiter of iteration progress. In production clusters, however, the dominant bottleneck may shift over time due to compute-side interference, hardware heterogeneity, stragglers, or data pipeline variability. In such cases, Gsched remains correct as a communication scheduler, but its end-to-end performance gain naturally diminishes as communication accounts for a smaller fraction of the iteration critical path. Therefore, Gsched should be viewed as complementary to compute-aware cluster schedulers rather than a replacement for them.

Fairness and Service Guarantees: Gsched relaxes strict flow-level fairness to accelerate synchronization-critical transmissions, but not into unbounded unfairness. Under admissible load, strict priority queuing serializes rather than starves flows, since the switch is work-conserving and aggregate demand stays within link capacity. Thus, lower-priority tasks receive eventual service within the scheduling window, although Gsched does not claim formal max-min fairness. Stronger production guarantees can be added through aging, priority promotion, or minimum-service reservation.

Bucketized Communication: Although our analysis uses communication phases or windows, Gsched is not limited to one monolithic communication block per iteration. Modern training systems often split communication into tensor buckets that may overlap with computation or one another. Gsched treats these transfers as finer-grained communication windows, or groups adjacent ones into short bursts, and applies the same priority-based ordering to active communication units on each bottleneck link.

B. Inputs and Objectives

Inputs: In Section V-A, we model each task J_i as a sequence of alternating computation and communication phases. For macroscopic scheduling decisions, we aggregate these into two core metrics: the total computation time $T_{c,i}$ and the equivalent total communication volume L_i per iteration. These

metrics capture the load characteristics essential for priority assignment.

Bottleneck Modeling: At each network bottleneck, multiple task flows arrive concurrently and exceed capacity. The fundamental scheduling choice is between fair queuing, which shares bandwidth and increases latency for all, and strict priority queuing, which serializes flows to minimize completion time for high-priority tasks.

Optimization Goal: Our objectives are based solely on DNN training task characteristics. We propose a system-centric metric, Average Transmission Efficiency (ATE), which aims to maximize network utilization under communication interleaving. Optimizing ATE ensures continuous data delivery, accelerates iteration cycles, and improves overall training performance.

C. Theoretical Analysis Roadmap

Recent systems such as Crux demonstrate excellent practical performance through GPU-intensity heuristics, but they also motivate a complementary theoretical question of how per-bottleneck scheduling can remain robust when global contention is decomposed across a large multi-tenant network. Theorems 1–3 show why interleaving and ATE-optimal ordering are beneficial, while Theorem 4 proves that the resulting decomposition remains robust against distributed conflicts, which is essential for large multi-tenant clusters.

Step 1 (Section V-B) Why Abandon Fair Scheduling?: Our analysis rigorously compares two communication paradigms: overlapping from fair scheduling and interleaving via active scheduling. We prove that overlapping causes significant performance loss due to bandwidth dilution and queuing delays. This establishes that active communication interleaving is essential for high performance (Theorem 1).

Step 2 (Section V-C) How to Leverage Inequality for Interleaving?: After establishing interleaving as the goal, we introduce a key mechanism: exclusive bandwidth occupancy. Using the exponential convergence theorem (Theorem 2), we show that by granting strict priority to a single task, the system deterministically breaks symmetric deadlocks and converges exponentially to the ideal interleaved state.

Step 3 (Section V-D) How to Achieve Optimal Order after Interleaving?: Communication interleaving serializes data flows, but the order matters. We derive a provably optimal scheduling policy for ATE maximization (Theorem 3). By using a dynamic decision function based on task attributes, we determine the locally optimal sequence that keeps the bottleneck saturated.

Step 4 (Section V-E) Is Distributed Implementation Feasible?: Maintaining a globally synchronized sequence in distributed systems is challenging. The Local Perturbation Theorem (Theorem 4) proves that minor local adjustments to the optimal sequence incur minimal performance loss. This guarantees that systems relying on loose coordination remain robust and near-optimal.

D. Design Overview

Section VI transforms the abstract theory into Gsched, a practical system that leverages standard commodity switch features.

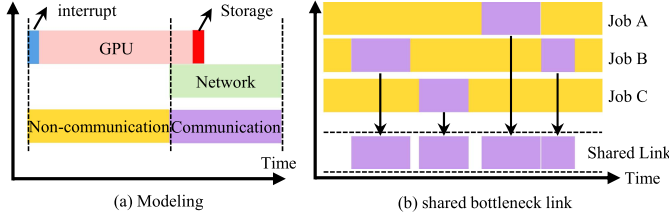


Fig. 3. (a) System modeling and (b) shared link.

The design decouples global decision-making from physical enforcement.

End-Host Logic: Instead of complex local rate control, end-hosts in Gsched focus on accurate information gathering. A Cold-Start Profiling mechanism captures the execution traces of new tasks, while a Path Pinning strategy ensures flow paths remain deterministic for centralized analysis.

Centralized Management: At the system level, a centralized server computes the optimal ATE sequence for each bottleneck. These decisions are enforced via DSCP priority mapping on commodity switches. Crucially, Gsched relies on physical network backpressure to naturally resolve conflicts when a flow traverses multiple bottlenecks, achieving robust distributed coordination consistent with Theorem 4.

V. THEORETICAL FOUNDATIONS

This section develops Gsched's theoretical foundation, covering the system model (Section V-A), the need for interleaving (Section V-B), priority-driven convergence (Section V-C), ATE-based ordering (Section V-D), perturbation stability (Section V-E), and distributed robustness (Section V-F).

A. System Model Formulation

In Fig. 3, a DNN training iteration includes fine-grained computation, transmission, memory, storage, synchronization, and interruption stages, some of which may overlap. For bottleneck-network scheduling, we abstract them into two states: a communication state, where a job injects traffic into a shared link, and a non-communication state, which aggregates all remaining periods. Each iteration is therefore modeled as alternating transmission and non-communication segments, with communication duration mainly determined by the exchanged data volume and available bandwidth under different parallel strategies.

In order to build a general theoretical model, we abstract this process by representing the task as a chain of dependencies. A DNN training job J_i is modeled as a tuple of alternating phases:

$$J_i = (T_{c,i}^1, L_i^1, T_{c,i}^2, L_i^2, \dots, T_{c,i}^K, L_i^K) \quad (1)$$

- 1) $T_{c,i}^k > 0$ represents the non-communication duration of the k -th stage within a single iteration.
- 2) $L_i^k > 0$ represents the transmission data volume of the k -th stage within a single iteration.
- 3) At each stage k , T and L may be different for one job.

Crucially, these stages exhibit strong temporal dependency: the $(k+1)$ -th non-communication stage cannot begin until the

k -th communication transmission L_i^k has fully completed. For scheduling analysis, we model each iteration as a sequence of alternating non-communication execution and network transmission, with no overlap between them.

We denote by $C_{i,n}^k$ the intrinsic, contention-free completion time of the k -th stage of task J_i on node n , namely the minimum time required in the absence of any network contention or blocking, and by $W_{i,n}^k$ the additional waiting time incurred due to network contention; consequently, the actual completion time of the stage is $C_{i,n}^k + W_{i,n}^k$.

For macroscopic analysis, we abstract away the fine-grained stage index k and use $T_{c,i}$ and L_i to represent the non-communication time and communication volume of a representative segment within an iteration of task J_i , respectively.

At the system level, we consider a set of N independent DNN training jobs $\mathcal{J} = \{J_1, J_2, \dots, J_N\}$ deployed in a data center network. The network contains multiple potential bottleneck links, and we assume all bottleneck links share an identical bandwidth capacity C . Different task flows may traverse different paths and contend for resources at these bottleneck points.

B. The Necessity of Interleaving

Building on the system model, we now focus on a single communication phase, in which each task J_i needs to transmit a data volume L_i over a shared bottleneck link of bandwidth C . Multiple tasks may attempt to transmit simultaneously, creating contention for the link. To understand how this competition affects performance, we examine the packet-level behavior of these transmissions. At this level, the order in which packets are served becomes important, which motivates the study of different queueing and scheduling strategies.

- 1) **Communication Overlapping:** The switch employs Fair Queueing (FQ). Bandwidth is dynamically divided among concurrent flows, causing packets from different tasks to interleave on the wire.
- 2) **Communication Interleaving:** The system employs Strict Priority Queueing (SPQ). High-priority packets are served first, while low-priority packets are buffered, effectively serializing the flows.

Lemma 1: Under the communication overlap paradigm, due to bandwidth dilution, the transmission duration $T_{overlap,i}$ for any task i is strictly greater than its ideal isolation time.

Proof: In a fair-queueing system, bandwidth is shared. As long as there is at least one other active flow competing, the instantaneous bandwidth $b_i(t)$ allocated to task i is less than C . Consequently, the inter-packet gaps are widened. The total completion time is:

$$T_{overlap,i} = \int \frac{1}{b_i(t)} dL_i > \int \frac{1}{C} dL_i = \frac{L_i}{C} \quad (2)$$

This inequality proves that FQ inherently prolongs the physical transmission process for every flow. ■

Lemma 2: Under the communication interleaving paradigm, enabled by buffering and strict prioritization, the highest-priority task k transmits at the full line rate C . Its duration reaches the physical lower bound.

Proof: Packets from different tasks arrive at the switch buffer asynchronously. Under Strict Priority Queuing, as long as the queue for task k (highest priority) is not empty, the scheduler transmits its packets immediately. Packets from lower-priority tasks are buffered in the queue and do not contend for the link. Thus, packets of task k are transmitted back-to-back on the wire. Task k effectively monopolizes the bandwidth:

$$T_{interleave,k} = \frac{L_k}{C} \quad (3)$$

With Lemma 1, we have $T_{interleave,k} < T_{overlap,k}$. ■

Theorem 1: In iterative computing systems, scheduled interleaving strictly outperforms concurrent overlap. By preventing bandwidth diversion to lower-priority tasks, it accelerates the completion of early tasks ($T^{PQ} < T^{FQ}$), enabling efficient computation pipelining.

Proof: We prove this by comparing the time required to finish the first k tasks in the schedule. Let $\mathcal{L}_k = \sum_{j=1}^k L_j$ be the cumulative data volume of these tasks.

Under interleaving, the scheduler concentrates 100% of the bandwidth on the high-priority sequence. The time to finish the first k tasks is exactly the time to transmit their data:

$$T_{finish,k}^{PQ} = \frac{\mathcal{L}_k}{C} \quad (4)$$

Under Overlapping, consider the same time point $t = \frac{\mathcal{L}_k}{C}$. The switch has transmitted a total data volume equal to $\mathcal{L}_k$. However, due to fair queuing, this transmitted volume is a mixture that includes packets from lower-priority tasks ($j > k$). Consequently, the first k tasks have not completed transmitting their full data $\mathcal{L}_k$ yet. Their actual completion time must be later:

$$T_{finish,k}^{FQ} > \frac{\mathcal{L}_k}{C} \quad (5)$$

For all tasks $k < N$, we have the inequality $T_{finish,k}^{PQ} < T_{finish,k}^{FQ}$. For the final task ($k = N$), the times are equal as the total volume is identical. This proves that SPQ allows the majority of tasks to detach from the network significantly earlier than in FQ, creating the necessary slack for computation pipelining. ■

Communication overlapping penalizes early tasks by diverting bandwidth to later tasks. Scheduled interleaving uses priority buffering to concentrate bandwidth, accelerating task completion without penalizing the total makespan. This establishes a deterministic, high-throughput pipeline essential for distributed training.

C. Accelerated Convergence Mechanism

Based on the theoretical advantage of communication interleaving discussed in the previous section, we now introduce a priority-based mechanism that accelerates the convergence of an N -task system from a state of link contention to conflict-free interleaving. By assigning a single task the highest transmission priority, the system quickly approaches the ideal interleaved state.

To analyze the dynamic evolution, we partition tasks into the priority task J_p with the highest transmission priority and the

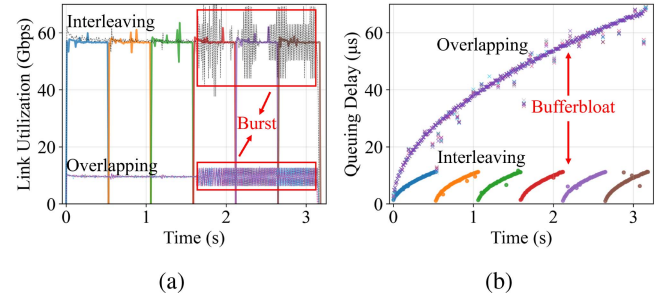


Fig. 4. (a) Throughput and (b) Queuing Delay in Communication Interleaving vs. Communication Overlapping.

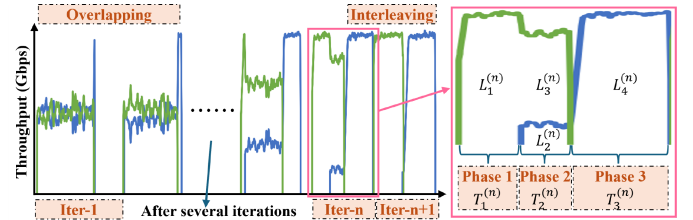


Fig. 5. Flows convergence to communication interleaving.

secondary-priority tasks J_{sec} comprising the remaining $N - 1$ tasks. We conducted experiments on a 100 Gbps link, where the total transmission volume of the secondary task set was equal to that of the highest priority task, and the throughput allocated to the high priority task was slightly higher than that of the task set.

Through flow-control adjustment, their throughputs are denoted as thr_p and thr_{sec} , respectively, subject to the link-saturation constraint $thr_p + thr_{sec} = C$. In Fig. 5, it enables precise tracking of the N -flows system under the priority strategy, which yields three distinct stages in each iteration's communication phase:

- 1) *Phase 1. Priority-Only Period (T_1):* J_p begins transmission while J_{sec} is still computing or its data has not arrived, fully occupying the link.
- 2) *Phase 2. Overlap and Contention (T_2):* Both J_p and J_{sec} transmit concurrently, sharing the link according to thr_p and thr_{sec} .
- 3) *Phase 3. Secondary-Only Period (T_3):* After J_p completes transmission, J_{sec} exclusively uses the link to finish its remaining data.

The system aims to minimize the time T_2 through priority control, ideally reducing it to zero after several iterations.

Assumption: For the closed form convergence law in Theorem 2, we use an idealized two group abstraction. The priority task J_p and the aggregated secondary task set J_{sec} have equal per iteration communication volume. Their throughput shares are quasi static during the overlap interval, with $thr_p + thr_{sec} \approx C$ and $thr_p > C/2$. These assumptions are used only to derive the exponential decay expression. In practice, unequal communication volumes and ECN regulated rate variations may change the decay constant, but they do not change the priority driven tendency that the overlap interval shrinks across iterations.

Theorem 2: In an N -flow system, if flow control guarantees the priority task J_p a throughput $thr_p > C/2$ and L_p is its per-iteration communication load, the communication overlap time $T_2^{(n)}$ with the secondary-priority set J_{sec} decays exponentially with iteration index n :

$$T_2^{(n)} = \frac{L_p}{thr_p} \left(\frac{C - thr_p}{thr_p} \right)^{n-1} \quad (6)$$

A detailed proof can be found in Appendix A.

D. Scheduling for ATE Maximization

Building on the previous analysis of communication interleaving and its collaborative mechanisms, we now address a higher-level scheduling problem: once interleaving is achieved and multiple tasks queue for network resources, in what order should they be served to optimize overall performance?

For heterogeneous DNN tasks, no single scheduling order is universally optimal; the notion of an optimal policy depends on the system-level objectives set by the administrator. This section exposes the inherent performance trade-offs and presents a unified theoretical framework encompassing scheduling strategies for different objectives.

Under communication interleaving, the scheduler must decide the serving order among heterogeneous tasks. Given parallelized computation but serialized transmission, our objective is to maximize link utilization, i.e., ensure continuous data delivery within each time window. This keeps the communication pipeline saturated, thereby shortening iteration cycles and improving overall training efficiency.

We propose ATE as a system-centric metric to evaluate the performance of a scheduling policy π . This metric quantifies the average fraction of time occupied by effective network transmission within a training iteration across all tasks. Our objective is to maximize $\eta(\pi)$, which is formally defined as:

$$\eta(\pi) = \frac{1}{N} \sum_{i=1}^N \frac{T_{t,i}}{T_{c,i} + T_{t,i} + W_i(\pi)} \quad (7)$$

ATE is a communication-centric metric intended for regimes where network contention dominates the iteration slowdown. It does not aim to replace compute-oriented objectives such as goodput or cluster-wide GPU efficiency, which are handled by complementary cluster schedulers.

Theorem 3: The optimal scheduling sequence π_{ATE}^* that maximizes the average transmission efficiency $\eta(\pi)$ for a set of tasks at a single bottleneck is generated by a greedy stepwise construction algorithm. At each step, a dynamic decision function Φ is used to choose the currently optimal task between two candidates a and b , where a is at position k and b at position $k+1$. The decision function is:

$$\Phi(a, b, W_{k-1}) = \left(\frac{T_{t,a}}{C_a + W_{k-1}} + \frac{T_{t,b}}{C_b + W_{k-1} + T_{t,a}} \right) - \left(\frac{T_{t,b}}{C_b + W_{k-1}} + \frac{T_{t,a}}{C_a + W_{k-1} + T_{t,b}} \right) \quad (8)$$

Proof: We prove the theorem by combining the Adjacent Pairwise Exchange method with a stepwise construction approach. We seek the optimal permutation π^* within the permutation space Π of the N tasks.

The first step establishes the local optimality condition. Consider any two adjacent tasks J_a and J_b in the priority scheduling sequence, with their accumulated waiting time before them denoted as W_{k-1} . We compare the contribution difference of the two possible orders to determine the local optimality criterion.

Order 1: $[a, b]$. The contribution to $\eta(\pi)$ by two tasks is:

$$\text{Contrib}(a, b) = \frac{T_{t,a}}{C_a + W_{k-1}} + \frac{T_{t,b}}{C_b + W_{k-1} + T_{t,a}} \quad (9)$$

Order 2: $[b, a]$. After swapping, the contribution is:

$$\text{Contrib}(b, a) = \frac{T_{t,b}}{C_b + W_{k-1}} + \frac{T_{t,a}}{C_a + W_{k-1} + T_{t,b}} \quad (10)$$

Define the contribution difference $\Delta = \text{Contrib}(a, b) - \text{Contrib}(b, a)$ as:

$$\Delta = \left(\frac{T_{t,a}}{C_a + W_{k-1}} + \frac{T_{t,b}}{C_b + W_{k-1} + T_{t,a}} \right) - \left(\frac{T_{t,b}}{C_b + W_{k-1}} + \frac{T_{t,a}}{C_a + W_{k-1} + T_{t,b}} \right) \quad (11)$$

Maximizing $\eta(\pi)$ requires $\Delta \geq 0$, which means the order $[a, b]$ is preferred. Since the sign of Δ depends dynamically on W_{k-1} and cannot be determined by intrinsic task attributes alone, the local optimal order is decided by the dynamic decision function $\Phi(a, b, W_{k-1}) = \Delta$. When $\Phi(a, b, W_{k-1}) \geq 0$, $[a, b]$ is locally optimal; otherwise, swap.

The second step proves global optimality through a greedy construction. The optimal sequence π^* can be obtained by a greedy construction algorithm with a worst-case complexity of $O(n^2)$, described as follows:

- 1) *Step 1* ($k = 1$): Compute pairwise $\Phi(i, j, 0)$ for all pairs. Select the task $\pi^*(1)$ that wins all comparisons ($\Phi \geq 0$) as the first element.
- 2) *Step k*: Assume the first $k-1$ tasks are determined, resulting in accumulated waiting time W_{k-1} . Compute pairwise $\Phi(i, j, W_{k-1})$ among the remaining unscheduled tasks. Select the winner $\pi^*(k)$ that satisfies the condition against all others as the k -th element.
- 3) Repeat until all tasks are scheduled.

We prove optimality by contradiction. Assume there exists a solution π_{opt} better than our constructed sequence π_g . Let position k be the first index where π_{opt} and π_g differ. At this step, given the same history W_{k-1} , π_{opt} selects task b , while our algorithm selects task a . By the definition of our greedy algorithm, task a must satisfy $\Phi(a, b, W_{k-1}) \geq 0$. This implies that placing a before b yields a higher or equal contribution than placing b before a . Therefore, the choice made by π_{opt} (selecting b) contradicts the local optimality condition established in Phase 1 and cannot result in a globally superior total efficiency. Thus, no better solution exists. ■

For any objective, the adjacent pairwise exchange principle derives the optimal task order by identifying the corresponding

local decision function, whose form determines complexity. If the sign of Δ depends only on static task attributes, ordering reduces to sorting with $O(n \log n)$ complexity. If Δ depends on dynamic preceding states such as W_{k-1} , the worst-case construction is $O(n^2)$, but the online sequence can be maintained efficiently with a heap or balanced data structure, where each insertion or deletion costs $O(\log n)$ and avoids quadratic overhead in practice.

E. Perturbation Stability in Scheduling

Theorem 3 establishes the optimal ordering rule for maximizing ATE at a single bottleneck. However, Gsched is deployed in a distributed network, where a flow may traverse multiple bottlenecks and encounter different local contention patterns. As a result, the globally preferred sequence computed at one bottleneck may not be realized exactly at another bottleneck. Such mismatch does not usually induce an arbitrary reshuffling of the entire schedule; instead, it appears as a local perturbation, where a task that should be served at position k is temporarily postponed by a competing task.

Therefore, the relevant theoretical question is not whether perturbation can be completely avoided, but whether the resulting deviation from the optimal sequence remains structurally benign. We emphasize that this statement is conditional for ATE. Since $\Phi(a, b, W_{k-1})$ depends on the fixed prefix through W_{k-1} , ATE does not imply global order consistency for arbitrary task sets. We use the theorem under a local fixed-prefix regime, where all feasible repairs share the same prefix and the relevant pairwise signs of Φ remain unchanged over the small waiting change induced by the repair. Outside this regime, the theorem should be read as a local approximation that explains why the smallest feasible exchange is the least disruptive repair.

Theorem 4 (Local Perturbation Stability): Let Π denote the permutation space of a task set, and let $f : \Pi \rightarrow \mathbb{R}$ be the scheduling objective to be maximized. Let $\pi^* = (\pi_1^*, \pi_2^*, \dots, \pi_N^*) \in \Pi$ be a globally optimal sequence.

Assume that f satisfies the following local order-consistency property over the repaired suffix. For any optimal sequence π^* , if two tasks x and y satisfy $x \prec y$ in π^* , then after fixing a common prefix that does not alter their feasibility, the optimal ordering of the remaining subproblem still prefers x before y . For ATE, this means that after the common prefix is fixed, the relevant pairwise comparisons induced by Φ keep the same sign within the small waiting range caused by the repair.

Suppose an external constraint prevents task π_k^* from being served before its successor π_{k+1}^* , so π_k^* must be postponed. Among all feasible repaired schedules that keep the prefix $(\pi_1^*, \dots, \pi_{k-1}^*)$ unchanged and move π_k^* to some later position $j > k$, the schedule obtained by a single adjacent exchange,

$$(\pi_1^*, \dots, \pi_{k-1}^*, \pi_{k+1}^*, \pi_k^*, \pi_{k+2}^*, \dots), \quad (12)$$

achieves the smallest performance degradation. Equivalently, if

$$\Delta(\tilde{\pi}) = f(\pi^*) - f(\tilde{\pi}), \quad (13)$$

then for every feasible repaired schedule $\pi^{(j)}$ with π_k^* moved to position $j > k$,

$$\Delta(\pi^{(k+1)}) \leq \Delta(\pi^{(j)}). \quad (14)$$

Proof: Let the optimal sequence be

$$\pi^* = (\dots, \underbrace{a}_k, \underbrace{b}_{k+1}, \underbrace{c}_{k+2}, \dots), \quad (15)$$

where the optimal precedence relation implied by π^* is

$$a \prec b \prec c. \quad (16)$$

By the order-consistency property, we also have

$$a \prec c \quad (17)$$

in the suffix subproblem once the common prefix before position k is fixed.

Now suppose an external perturbation forces task a to be postponed. Consider two feasible repairs:

$$\pi'_1 = (\dots, b, a, c, \dots), \quad (18)$$

$$\pi'_2 = (\dots, b, c, a, \dots). \quad (19)$$

The two repaired sequences share the same fixed prefix $(\dots, b)$. Therefore, their difference is entirely determined by the remaining suffix order: π'_1 retains the relative order (a, c) , while π'_2 reverses it into (c, a) .

Since the suffix subproblem still preserves the optimal order between unaffected tasks and $a \prec c$ in that subproblem, placing a before c is optimal whenever both remain feasible after the same prefix. Hence,

$$f(\pi'_1) \geq f(\pi'_2), \quad (20)$$

or equivalently,

$$\Delta(\pi'_1) \leq \Delta(\pi'_2). \quad (21)$$

Thus, postponing a by one position causes no more degradation than postponing it by two positions.

We next generalize this argument. Let $\pi^{(j)}$ denote the repaired sequence obtained by moving a from position k to position $j > k$. Such a move can always be decomposed into a sequence of adjacent exchanges:

$$\pi^{(k+1)}, \pi^{(k+2)}, \dots, \pi^{(j)}, \quad (22)$$

where at step r , task a is swapped only with its current immediate successor. At each step, the same argument as above applies: exchanging a with only the next task preserves as much of the optimal suffix order as possible, whereas moving a further back necessarily introduces at least one additional inversion against a task that should remain behind a in the optimal structure. Therefore, each extra displacement cannot reduce the loss relative to the smaller displacement.

By induction on $(j - k)$, we obtain

$$f(\pi^{(k+1)}) \geq f(\pi^{(j)}), \quad \forall j > k + 1, \quad (23)$$

which proves that the adjacent exchange yields the minimum degradation among all feasible postponements of a . ■

Theorem 4 has two implications. First, it establishes a local stability property: when an external conflict makes the original order infeasible, the smallest structural deviation causes the smallest objective loss. Second, it bridges the single-bottleneck ordering of Theorem 3 and the distributed realization of Gsched. Disagreements among bottlenecks need not trigger global rescheduling; each conflict can be treated as a local perturbation repaired by the smallest feasible exchange.

This interpretation is important for ATE maximization but remains local. Theorem 3 generates the ATE sequence through repeated local decisions under the dynamic comparison function $\Phi(a, b, W_{k-1})$. With a fixed common prefix, Gsched compares feasible repairs under the same waiting state. If the relevant signs of Φ remain stable within the small waiting perturbation, the suffix order is locally consistent. Otherwise, adjacent repair is used as a local approximation rather than an unconditional optimality guarantee.

When a downstream bottleneck or congestion-control event delays a task, the realized schedule differs from the single-bottleneck optimum only through sparse local displacements. By Theorem 4, resolving each displacement as an adjacent repair is least harmful. If a task must be postponed by multiple positions, the least-loss repair is a sequence of adjacent exchanges rather than a direct relocation, so performance loss grows gradually with the number of local perturbations. This justifies Gsched's design choice to avoid a globally synchronized total order and instead preserve the ATE-oriented structure through local priority enforcement, minimal repairs, and congestion-control feedback.

F. Distributed Consistency and Performance Robustness

While Theorem 3 provides the optimal ordering for a single bottleneck, a flow in a distributed network may traverse multiple bottlenecks, each potentially calculating a different optimal sequence. A global optimization solving for all links simultaneously is computationally intractable and unscalable. Instead, Gsched computes local optima using Theorem 3 and resolves conflicts via congestion-driven rate regulation. Critically, Theorem 4 bridges the gap between local and global performance. It guarantees that when a conflict forces a local priority swap, the impact on the global objective function is minimal. This theoretical stability justifies our design choice of using decoupled, per-link scheduling.

Consider a flow F_1 traversing two bottlenecks, Link A and Link B .

- At Link A , the local scheduler assigns F_1 the highest priority based on its local optimal sequence $\pi_A^* = (F_1, F_3, \dots)$.
- At Link B , due to different contention patterns, the scheduler assigns a higher priority to another flow F_2 , resulting in a sequence $\pi_B^* = (F_2, F_1, \dots)$.

Physical Consequence: When F_1 contends at Link B , congestion control reacts to queue buildup by reducing F_1 's sending rate, so F_1 consumes only part of the bandwidth at upstream Link A . Since the switch is work-conserving, the remaining bandwidth is reallocated to the next ready task F_3 , improving link utilization. This behavior complements ECN-based data-center congestion control such as DCQCN and DCTCP. The transport layer adjusts rates from queue feedback, while Gsched

controls the relative service order, forming a cooperative loop where congestion control absorbs downstream contention and Gsched preserves the intended priority structure.

Equivalent Scheduling View: From the perspective of Link A , the actual transmission order deviates from its calculated optimum π_A^* . F_1 , originally scheduled first, is effectively postponed or interleaved with F_3 due to external constraints from Link B . This creates a perturbed execution sequence π'_A .

Robustness Analysis: According to Theorem 4, when the local order-consistency condition holds under the fixed prefix, the degradation caused by such local displacements is minimized by the adjacent-repair view. For ATE, this is a local guarantee when the relevant signs of Φ remain stable. Otherwise, it serves as an approximation that explains why sparse local conflicts usually cause limited degradation rather than global reordering.

This analysis shows that independent, distributed scheduling agents can operate without full global coordination. Local inconsistencies are naturally tolerated, and overall performance remains near-optimal. These insights guide the design of our distributed scheduling framework in the next section.

VI. DESIGN AND IMPLEMENTATION

This section presents Gsched's design and implementation, including priority-based forwarding (Section VI-A), profiling and ATE optimization (Section VI-B), distributed coordination (Section VI-C), and scalability analysis (Section VI-D).

A. System Architecture and Priority-Based Forwarding

The Gsched architecture has three components. Commodity switches provide data forwarding and enforce DSCP-based strict priority scheduling. The Centralized Scheduling Server maintains the topology state, computes the scheduling sequence for each bottleneck link, and distributes priority instructions. End-Host Agents run on training servers to intercept NCCL operations, identify flows, collect performance data, and dynamically mark packet priorities.

To achieve fine-grained scheduling control, we map the eight physical queues of the switch, denoted as Queue 0 to Queue 7 where Queue 0 has the highest priority, as follows.

- *Queue 0 for Critical Control:* This queue is dedicated to transmitting Acknowledgments, Congestion Notification Packets, and the control signaling of Gsched itself. Assigning it the highest priority ensures that the system control plane remains non-blocking under deployable workload conditions. In extreme overload scenarios, end-to-end congestion control provides adaptive rate throttling to prevent persistent blocking.
- *Queue 1 for Cold-Start:* New tasks first enter this queue for several warm-up iterations, allowing the agent to collect traffic characteristics before normal scheduling.
- *Queue 2 to Queue 6 for Tiered Scheduling:* These queues serve the top 5 tasks in the ATE sequence π^* , with higher-ranked tasks mapped to higher-priority queues. If a higher-ranked task is blocked downstream, the switch immediately serves the next ready task, preserving orderliness and utilization.

- *Queue 7 for Buffer-Constrained Waiting:* This queue holds tasks outside the top tiers and utilizes aggressive ECN marking to shift backlog from switch buffers to end-host. The work-conserving switch grants immediate bandwidth to these tasks once higher queues are drained while the centralized controller promotes them to high-priority queues in the subsequent cycle. This mechanism maximizes link utilization and guarantees eventual task completion without starvation.

B. Scheduling Workflow From Profiling to ATE Optimization

The operation of Gsched follows a closed-loop process comprising four phases, including flow identification, cold-start profiling, centralized decision-making, and priority enforcement.

Flow Identification and Dynamic Path Pinning: Deep learning training typically relies on persistent connections such as RDMA Queue Pairs. To enable the centralized server to construct a deterministic dependency graph, the flow's physical path must be predictable. However, statically fixing a source port for the entire session may lead to hash polarization.

Gsched addresses this via a per-iteration dynamic pinning strategy.

- 1) *Periodic Probing:* At the boundary of each iteration, the End-Host Agent initiates lightweight probes using a set of candidate UDP source ports.
- 2) *Optimal Selection:* The centralized server correlates these ports with physical paths via ECMP logic to identify the path with the least background load.
- 3) *Deterministic Execution:* The server instructs the Agent to pin this optimal UDP source port for the duration of the current iteration.

This mechanism guarantees that within each scheduling window, the physical path remains deterministic for ATE optimization, while enabling adaptive rotation across iterations to mitigate long-term collisions.

Cold-Start Profiling: When a new training task J_i starts, it is mapped to Queue 1 by default. The task runs for a configurable warm-up period, for example, 10 iterations. During this period, the end-host agent captures the fine-grained execution trace of the task. Specifically, it records the duration of each non-communication phase $T_{c,i}^k$ and the data volume of each communication phase L_i^k within a single iteration, along with the 5-tuple information used. Upon completion of profiling, the agent reports this sequence of phases to the central server and automatically downgrades the traffic of the task to Queue 7 to await further scheduling instructions.

Centralized Decision Making: The scheduling server maintains a global view of the network topology. First, based on the candidate ports reported by the agent and the switch's ECMP hashing logic, the server selects and locks the optimal physical path for each data flow to avoid hotspots. Second, the server aggregates the load demand on every link along these determined paths to identify bottleneck links where utilization approaches saturation. Finally, for each identified bottleneck link, the server executes the ATE maximization algorithm to compute the optimal ranking sequence π^* for the flows.

Token Dispatch and Priority Enforcement: Based on the calculation results, the server sends corresponding priority tokens to the top 5 ranked tasks. Upon receiving a token, the end-host agent modifies the Traffic Class field of the QP by invoking RDMA verbs interfaces. The value of this field maps directly to the DSCP value in the IP packet header, thereby injecting data packets into the corresponding queues from Queue 2 to Queue 6 for transmission.

C. Distributed Coordination

In a distributed network, a flow may traverse multiple bottlenecks whose local rankings conflict. Gsched avoids global locking and assigns the flow the highest priority it obtains across all traversed bottlenecks. For example, if a flow is ranked for Queue 2 at Link A and Queue 4 at Link B, it is assigned Queue 2.

The physical execution is regulated by network backpressure and work-conserving switching. If the flow is blocked by a higher-priority task at a downstream link, its packets trigger ECN marking or RTT inflation, and end-host congestion control throttles its sending rate. The released upstream bandwidth is then immediately used by the next ready task in the local priority sequence. This dynamic yielding implements the adjacent-repair behavior analyzed in Theorem 4, allowing Gsched to tolerate distributed conflicts without maintaining a globally synchronized total order.

D. Scalability and Control Efficacy

Addressing the challenges of managing large-scale clusters, Gsched ensures scalability and responsiveness through parallel decomposition and latency hiding.

Parallel Decomposition: Gsched scales by decomposing scheduling per bottleneck link instead of solving one monolithic cluster-wide optimization. The central server runs the ordering algorithm independently for each identified bottleneck. Since AI training traffic is usually confined within task groups such as AllReduce, and further constrained by placement, the number of concurrent flows N on a link is typically below 20. The ordering cost is $O(N \log N)$ for static decision functions, and remains microsecond-level for such small N even with dynamic state. With multi-core parallelism, one scheduling server can process thousands of bottlenecks with modest control overhead. Each iteration only requires lightweight profiling reports and a few priority-token updates for top-ranked tasks, rather than per-packet or globally synchronized control actions.

Latency Hiding via Overlap: Gsched hides control-plane latency by exploiting the alternating computation and communication phases of DNN training. During non-communication phases, the server computes scheduling decisions and dispatches priority tokens in advance. Since computation usually exceeds the control-plane RTT, updated DSCP mappings reach end hosts before the next communication phase, aligning packet-level switch scheduling with the flow-level plan. Unlike time-division schemes that require microsecond-level clock synchronization, Gsched relies on relative priority ordering. With work-conserving SPQ, execution jitter does not prevent the link

TABLE II
DNN WORKLOAD DEPLOYMENTS IN DIFFERENT SCENARIOS

Scenarios	Deployed Tasks
Heterogeneous	GPT-2, ResNet, VGG16, VGG19, CamemBERT, RoBERTa, BERT
Homogeneous	GPT-2 $\times$ 7
Semi-homogeneous	GPT-2 $\times$ 3, BERT $\times$ 2, VGG16 $\times$ 2
Highly-diverse	GPT-3, GPT-2, BERT, VGG16

from serving the next ready flow. Thus, scheduling latency is largely removed from the communication critical path, and scaling mainly increases the number of independent bottleneck decisions rather than the per-link reaction latency seen by each task.

VII. EVALUATION

This section evaluates Gsched through the experimental setup (Section VII-A), homogeneous workloads (Section VII-B), heterogeneous workloads (Section VII-C), semi-homogeneous workloads (Section VII-D), highly diverse workloads (Section VII-E), convergence analysis (Section VII-F), and evaluation scope (Section VII-G).

A. Experimental Setup

Hardware Setup: We use a physical cluster of 8 servers connected via a 2-tier Leaf-Spine topology comprising 4 spine switches and 4 leaf switches (Cisco 9500-series). We set the per-queue buffer size to 500 KB to accommodate traffic bursts, while configuring ECN thresholds at $K_{min} = 30$ KB and $K_{max} = 65$ KB to maintain low queuing delay. To emulate realistic congestion, we configure a 2:1 oversubscription ratio at the leaf layer. Specifically, each leaf switch connects downlink to 2 servers aggregating 800 Gbps bandwidth and uplink to 4 spine switches aggregating 400 Gbps bandwidth. The 32 total RDMA endpoints run RoCEv2 with ECN enabled, controlled by a dedicated out-of-band scheduling server. To determine the workloads, we collected raw data from real distributed training sessions on a reference cluster of dual servers equipped with 4 NVIDIA RTX 4090 GPUs, each with 48 GB memory, for replay simulation.

DNN Workloads: We experiment with 8 representative DNN models: GPT-2 [55], GPT-3 [12], ResNet50 [27], VGG16, VGG19 [61], BERT [17], RoBERTa [41], and CamemBERT [45]. All models have equal arrival probability. We categorize deployments into four types of scenarios to study inter-job scheduling performance in Table II. Homogeneous refers to a scenario in which all tasks are of the same model type. Heterogeneous refers to a scenario in which all tasks are of different model types. Semi-homogeneous refers to a scenario in which some tasks are identical while others differ. Highly-diverse refers to a variant of the heterogeneous scenario in which tasks with differences in iteration time and data transfer volume are included to create maximal diversity.

Metrics: Waiting time refers to the time during a training iteration in which a device remains idle after completing its computation, typically while waiting for gradient aggregation, parameter synchronization, or for other devices to reach the

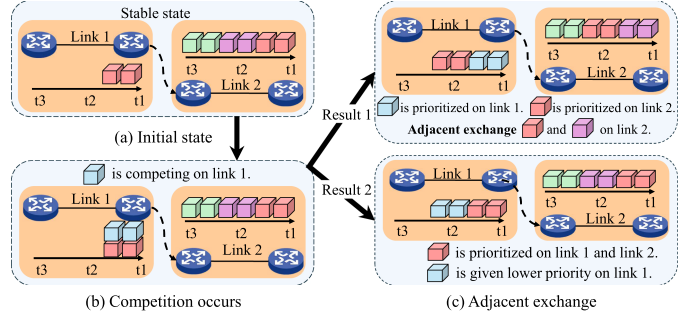


Fig. 6. Illustration of local perturbation and adjacent repair. A task that is preferred in the globally optimal sequence may be temporarily postponed by a downstream constraint. The least-disruptive repair is an adjacent exchange rather than a large reordering.

same iteration. Link utilization refers to the Probability Density Function (PDF) calculated over a sliding window from [0,2] seconds with steps of 0.01 seconds. GPU utilization refers to the fraction of each iteration that is spent on computation. Iteration time refers to the time required to complete one training iteration of a DNN, including forward propagation, backward propagation, multiple communication steps, gradient aggregation, and synchronization across devices. Convergence time refers to the number of iterations required for the overall iteration time across all tasks to stabilize.

Baseline: We compare Gsched with five baselines. DCTCP [6] represents ECN-based congestion control for flow-level scheduling. MLTCP [57] is an end-host rate-control method for inter-job communication interleaving in DNN training. SRTF [59] schedules flows by shortest remaining time, while pFabric [4] prioritizes packets by remaining flow size to minimize FCT. Crux [13] is a GPU-intensity-aware scheduler that mitigates inter-job contention to improve GPU utilization. We also report Upper/Lower Bound results, where tasks are completely non-conflicting and incur no waiting.

Source of performance gains: Gsched's performance gains mainly come from two coupled effects. First, strict-priority interleaving suppresses bandwidth dilution and queue buildup by serializing competing communication windows at each bottleneck. Second, ATE-based ordering uses each task's computation/communication profile to choose a service order that better matches the training pipeline. These decisions reduce communication stalls, keep bottleneck links more continuously occupied, and therefore improve iteration time, waiting latency, link utilization, and GPU utilization in a consistent way.

B. Homogeneous Scenario

Iteration Time: In Fig. 7(a), the Cumulative Distribution Function (CDF) of iteration time illustrates the performance distribution of the algorithms across seven homogeneous GPT-2 tasks. In Table III, Gsched achieves an iteration time of 210.1 ms, only 0.05% above the experimentally achievable lower bound of 210.0 ms, approaching the minimum achievable time constrained by physical limitations. In comparison, SRTF reaches 239.9 ms, exceeding the lower bound by approximately 14.2%,

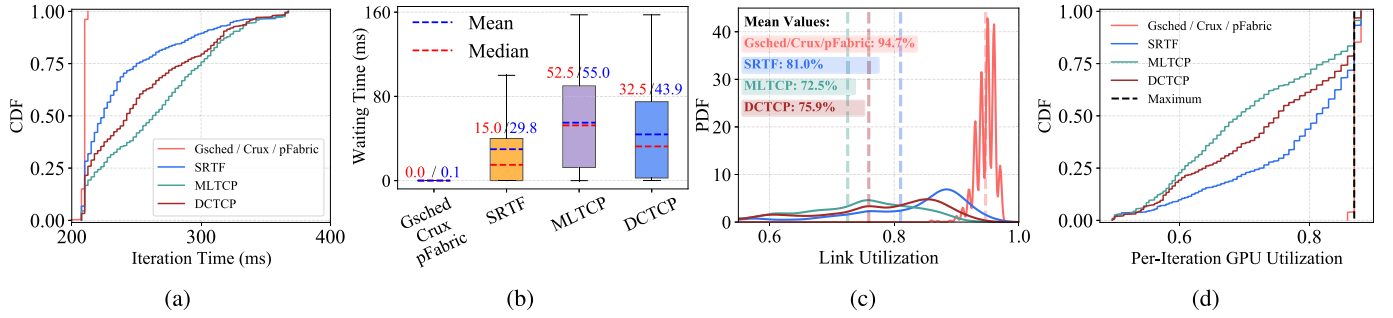


Fig. 7. Overall performance of the tasks for homogeneous scenario: (a) iteration times, (b) waiting times, (c) link utilization, and (d) GPU utilization.

TABLE III
OVERALL PERFORMANCE OF DIFFERENT ALGORITHMS UNDER DIFFERENT SCENARIOS

Algorithms	Homogeneous				Heterogeneous				Semi-homogeneous				Highly Diverse			
	GPU (%)	Iteration (ms)	Link (%)	Waiting (ms)	GPU (%)	Iteration (ms)	Link (%)	Waiting (ms)	GPU (%)	Iteration (ms)	Link (%)	Waiting (ms)	GPU (%)	Iteration (ms)	Link (%)	Waiting (ms)
Gsched	86.3	210.1	94.7	0.1	85.9	239.4	68.9	11.8	85.4	234.2	73.3	11.3	78.3	204.8	62.2	10.3
Crux	86.3	210.1	94.7	0.1	80.9	254.9	62.6	25.9	85.4	233.7	73.8	10.3	70.9	224.1	57.5	32.8
pFabric	86.3	210.1	94.7	0.1	81.5	253.0	63.4	24.1	84.5	236.6	72.1	13.0	71.3	222.2	58.2	30.6
SRTF	77.8	239.9	81.0	29.8	83.2	247.8	66.4	20.1	82.2	244.3	70.8	20.8	77.0	210.3	59.6	14.2
MLTCP	70.7	265.3	72.5	55.0	82.6	249.7	65.8	21.9	82.0	244.8	70.5	21.3	76.9	210.7	59.7	14.4
DCTCP	73.8	254.2	75.9	43.9	83.4	247.3	66.3	19.6	81.8	245.5	70.0	22.1	76.8	210.7	59.6	14.5
Upper/Lower Bound	87.0	210.0	95.8	0.0	89.8	228.0	70.6	0.0	89.1	223.3	75.6	0.0	81.9	195.7	63.4	0.0

while MLTCP and DCTCP deviate by 26.3% and 21.1%, respectively. The CDF curves indicate that Gsched, Crux, and pFabric exhibit highly overlapping and steeply rising curves, demonstrating that they can consistently maintain iteration times close to the physical lower bound in homogeneous environments. By contrast, SRTF, MLTCP, and DCTCP display more dispersed iteration time distributions.

Waiting Latency: In Fig. 7(b), the box plot of waiting times highlights significant differences among the algorithms. Gsched achieves an average waiting time of 0.1 ms, nearly reaching ideal zero latency. Relative to Gsched, the deviation multiples of the other algorithms are approximately: SRTF 298 $\times$, MLTCP 550 $\times$, and DCTCP 439 $\times$. Other algorithms show substantial degradation, with SRTF at 29.8 ms, MLTCP at 55.0 ms, and DCTCP at 43.9 ms. The box plot further illustrates that Gsched's latency distribution is highly concentrated with no outliers, whereas MLTCP and DCTCP exhibit peaks up to 80–100 ms, reflecting the queuing delays inherent in traditional congestion control under DNN synchronous communication.

Link Utilization: In Fig. 7(c), the PDF of link utilization reflects bandwidth efficiency. Gsched reaches 0.947, close to the experimentally achievable upper bound of 0.958, corresponding to 98.9% proximity. In contrast, SRTF achieves 0.81, MLTCP 0.725, and DCTCP 0.759. Compared to the next-best method, SRTF, Gsched leads by approximately 3.1 percentage points in proximity to the upper bound, demonstrating clear advantages in bandwidth utilization.

GPU Utilization: In Fig. 7(d), the CDF of GPU utilization demonstrates the efficiency of GPU usage. Gsched achieves 0.863, close to the experimentally achievable upper bound of

0.87. SRTF reaches 0.778, MLTCP 0.707, and DCTCP 0.738. Compared to the worst performer, MLTCP, Gsched leads by 17.9%, achieving approximately 1.22 $\times$ performance improvement. The CDF curves show that Gsched converges rapidly around 0.86, whereas other algorithms are more dispersed, confirming the effectiveness of the communication interleaving mechanism in maintaining high GPU utilization.

C. Heterogeneous Scenario

Iteration Time: In a heterogeneous environment, the CDF of iteration time demonstrates the algorithms' ability to handle diverse tasks. In Fig. 8(a), Gsched achieves an average iteration time of 239.4 ms (deviation $\sim 5.0\%$ relative to the experimentally achievable lower bound of 228.0 ms). In comparison, Crux reaches 254.9 ms, pFabric 253.0 ms, and SRTF 247.8 ms. The CDF curves show that Gsched's distribution is the most left-shifted and steepest, whereas other methods exhibit longer tails, indicating higher iteration-time consistency under heterogeneous workloads. This advantage comes from ATE-based ordering, which better aligns communication service with each task's computation phase and reduces unnecessary waiting before synchronization-critical transmissions.

Waiting Latency: In Fig. 8(b), the box plot of waiting times highlights differences in task scheduling fairness. Gsched maintains an average waiting time of 11.8 ms. In comparison, Crux reaches 25.9 ms, pFabric 23.6 ms, and SRTF 20.1 ms. The box plot shows that Gsched's median is around 5.0 ms with a small interquartile range, whereas Crux and pFabric exhibit extreme values up to 40–50 ms, and SRTF, MLTCP, and DCTCP show

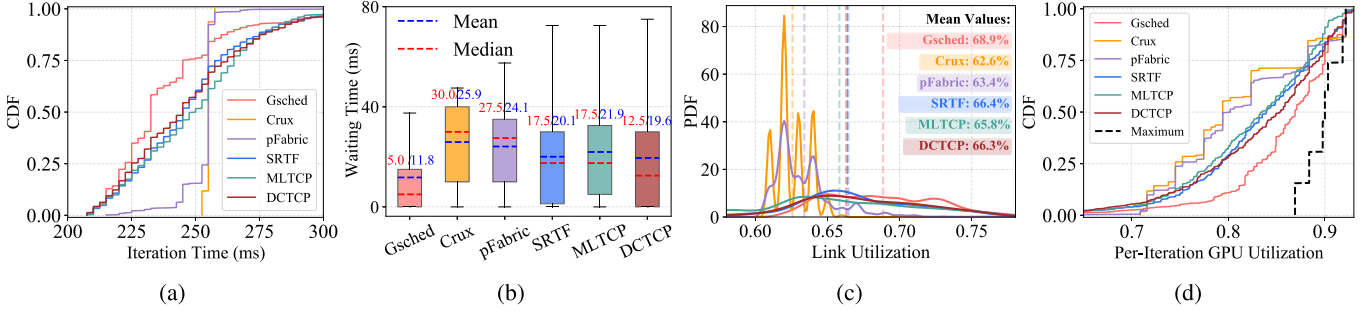


Fig. 8. Overall performance of the tasks for heterogeneous scenario: (a) iteration times, (b) waiting times, (c) link utilization, and (d) GPU utilization.

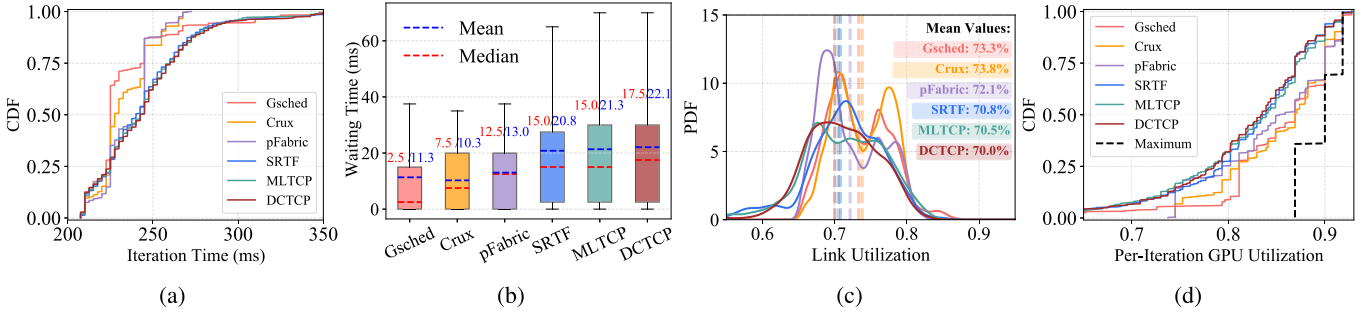


Fig. 9. Overall performance of the tasks for semi-homogeneous scenario: (a) iteration times, (b) waiting times, (c) link utilization, and (d) GPU utilization.

more dispersed latency distributions, with maximum delays reaching 70-80 ms. This indicates that other methods tend to produce unbalanced waiting times under heterogeneous workloads.

Link Utilization: In Fig. 8(c), the PDF of link utilization reflects bandwidth efficiency in heterogeneous environments. Gsched achieves 0.689, close to the experimentally achievable upper bound of 0.706, with a proximity of approximately 97.6%. In contrast, Crux, pFabric, and SRTF achieve proximities of 88.7%, 89.8%, and 94.1%, corresponding to deviation multiples of approximately $3.3\times$, $2.8\times$, and $1.8\times$ relative to Gsched, showing that Gsched improves bandwidth utilization in this heterogeneous setting. The PDF curve shows that Gsched's peak is around 0.69 and highly concentrated, while the distributions of other methods are flatter and more dispersed, demonstrating Gsched's stability in heterogeneous task scheduling.

GPU Utilization: In Fig. 8(d), the CDF of GPU utilization demonstrates the coordination efficiency of computational resources in heterogeneous environments. Gsched achieves 0.859, close to the experimentally achievable upper bound of 0.898, with a proximity of approximately 95.7%, among the higher utilization results in this setting. Crux and pFabric achieve proximities of 90.1% and 90.8%, corresponding to deviation multiples of approximately $2.3\times$ and $1.9\times$, while SRTF is about $1.5\times$. The CDF curves show that Gsched rises sharply around 0.86, whereas traditional congestion control methods MLTCP and DCTCP curves shift to the right, with GPU utilization proximities of 92.0% and 92.9%, confirming Gsched's effectiveness in efficiently coordinating GPU resources under heterogeneous environments.

D. Semi-Homogeneous Scenario

Iteration Time: In a semi-homogeneous environment containing 3 GPT-2, 2 BERT, and 2 VGG16 models, the CDF of iteration time demonstrates the algorithms' performance under moderate task diversity. In Fig. 9(a), Gsched achieves an average iteration time of 234.2 ms, with a deviation of 4.9% relative to the experimentally achievable lower bound of 223.3 ms, approaching the minimum achievable iteration time. Crux achieves 233.7 ms with a deviation of 4.6%, slightly better than Gsched in this common semi-homogeneous regime, consistent with Crux's strong GPU-intensity heuristic. Nevertheless, Gsched remains competitive and demonstrates clearer advantages in the highly-diverse case (204.8 ms vs. Crux's 224.1 ms), where ATE-optimal ordering better handles extreme heterogeneity. The deviations of pFabric, SRTF, MLTCP, and DCTCP are 5.9%, 9.4%, 9.6%, and 9.9%, corresponding to approximately $1.2\times$, $1.9\times$, $1.9\times$, and $2.0\times$ that of Gsched. The CDF curves show that Gsched and Crux distributions almost overlap and are most left-shifted, rapidly converging near 235 ms, while other methods extend to 280–300 ms, indicating that task similarity in semi-homogeneous scenarios helps multiple algorithms achieve comparable performance.

Waiting Latency: In Fig. 9(b), the box plot of waiting times shows that Gsched maintains an average waiting time of 11.3 ms with a median of approximately 7.5 ms. Crux achieves 10.3 ms, slightly lower than Gsched. The waiting times of pFabric, SRTF, MLTCP, and DCTCP are 13.0 ms, 20.8 ms, 21.3 ms, and 22.1 ms, corresponding to approximately $1.15\times$, $1.84\times$, $1.88\times$, and $1.95\times$ that of Gsched. The box plot shows that Gsched and Crux distributions are concentrated, whereas traditional methods

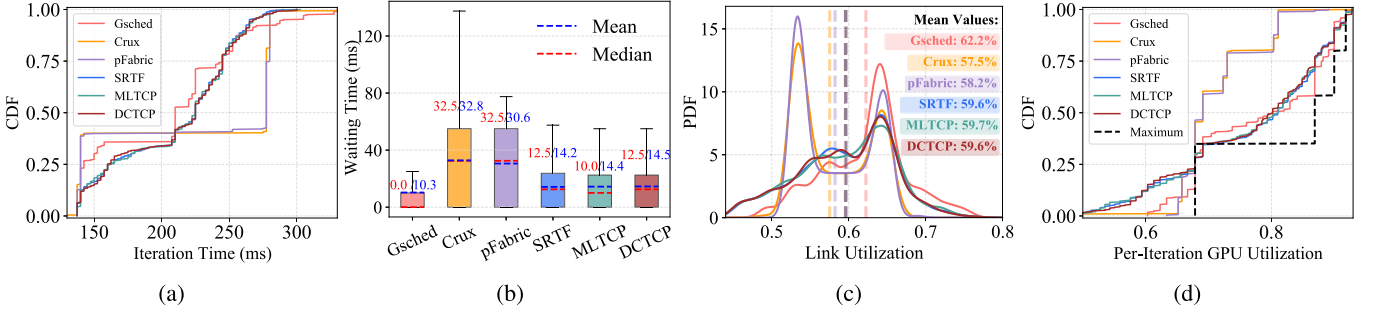


Fig. 10. Overall performance of the tasks for highly diverse heterogeneous scenario: (a) iteration times, (b) waiting times, (c) link utilization, and (d) GPU utilization.

exhibit more dispersed latency with extreme peaks, reflecting the moderate scheduling complexity in semi-homogeneous task combinations.

Link Utilization: In Fig. 9(c), the PDF of link utilization shows that Gsched achieves 0.733, close to the experimentally achievable upper bound of 0.756, with a deviation of approximately 3.0%. Crux achieves 0.738 with a deviation of 2.4%, slightly better than Gsched. The deviations of pFabric, SRTF, MLTCP, and DCTCP are 4.6%, 6.3%, 6.7%, and 7.4%, corresponding to approximately 1.5 $\times$, 2.1 $\times$, 2.2 $\times$, and 2.5 $\times$ that of Gsched. The PDF curves show multi-peak distributions for all algorithms, with Gsched and Crux peaks more pronounced and concentrated in the high-utilization region.

GPU Utilization: In Fig. 9(d), the CDF of GPU utilization shows that Gsched achieves 0.854, close to the experimentally achievable upper bound of 0.891, with a deviation of approximately 4.2%. Crux also achieves 0.854 with a deviation of 4.2%. The deviations of pFabric, SRTF, MLTCP, and DCTCP are 5.3%, 7.7%, 8.0%, and 8.2%, corresponding to approximately 1.3 $\times$, 1.8 $\times$, 1.9 $\times$, and 2.0 $\times$ that of Gsched. The CDF curves show that Gsched and Crux rise sharply near 0.85, while traditional methods rise more slowly, confirming the effectiveness of Gsched in efficiently coordinating GPU resources in semi-homogeneous environments.

E. Highly Diverse Heterogeneous Scenario

Iteration Time: In the highly diverse scenario with GPT-3, GPT-2, BERT, and VGG16, models differ substantially in computation time and communication volume. As shown in Fig. 10(a), Gsched achieves an average iteration time of 204.8 ms, about 4.6% above the experimentally achievable lower bound of 195.7 ms. In comparison, Crux reaches 224.1 ms, pFabric reaches 222.2 ms, and SRTF reaches 210.3 ms, with deviations of approximately 3.18 $\times$, 2.2 $\times$, and 1.7 $\times$ that of Gsched, respectively. The CDF curve of Gsched is more left-shifted and steeper, while other methods show longer tails. This indicates that ATE-based ordering better handles cases where computation times and communication volumes differ substantially, reducing idle gaps and long waiting tails.

Waiting Latency: In Fig. 10(b), the box plot reveals scheduling challenges caused by task imbalance. Gsched maintains an

average waiting time of 10.3 ms with a median of approximately zero, the only method keeping waiting times at single-digit millisecond levels. In comparison, Crux averages 32.8 ms (approximately 3.18 $\times$), pFabric 30.6ms (2.97 $\times$), SRTF 14.2ms (1.38 $\times$), and MLTCP and DCTCP 14.4ms and 14.5ms (both approximately 1.4 $\times$). The box plot shows that Gsched's latency distribution is highly concentrated with minimal variance, whereas Crux and pFabric exhibit larger tail values in this highly diverse case, reflecting that fixed priority proxies can be less aligned with the iteration-level objective when task sizes differ substantially.

Link Utilization: In Fig. 10(c), the PDF of link utilization reflects the complexity of bandwidth allocation under extreme heterogeneity. Gsched reaches 0.622, close to the upper bound of 0.634, with a near-optimal performance of approximately 98.1%. The corresponding performances of other methods are: Crux 90.7% (7.4% below), pFabric 91.8% (6.3% below), SRTF 94.0% (4.1% below), MLTCP 94.2% (3.9% below), and DCTCP 94.0% (4.1% below). The PDF curves show bimodal distributions for all methods, reflecting scheduling differentiation between large GPT-3 tasks and smaller tasks, but Gsched's main peak is more pronounced and located in the high-utilization region, demonstrating stable bandwidth scheduling.

GPU Utilization: In Fig. 10(d), the CDF of GPU utilization shows the challenge of coordinating computational resources under highly heterogeneous workloads. Gsched achieves 0.783 GPU utilization, corresponding to 95.6% of the upper bound of 0.819, outperforming other methods. Crux, pFabric, and SRTF reach 86.6% of bound (2.3 $\times$ deviation), 87.0% (1.9 $\times$), and 94.0% (1.5 $\times$), while MLTCP and DCTCP achieve 93.9% and 93.8% (both approximately 1.5 $\times$). The CDF curve shows Gsched rising sharply after a plateau near 0.78, indicating its ability to effectively adjust computational resource allocation when handling extreme heterogeneous tasks.

F. Convergence Performance

Completely Homogeneous Scenario: In the homogeneous scenario, all four algorithms show convergence in iteration time, but at different speeds in Fig. 11. Gsched stabilizes within the initial iterations, SRTF fluctuates early and then converges, DCTCP converges slightly slower than SRTF, and MLTCP

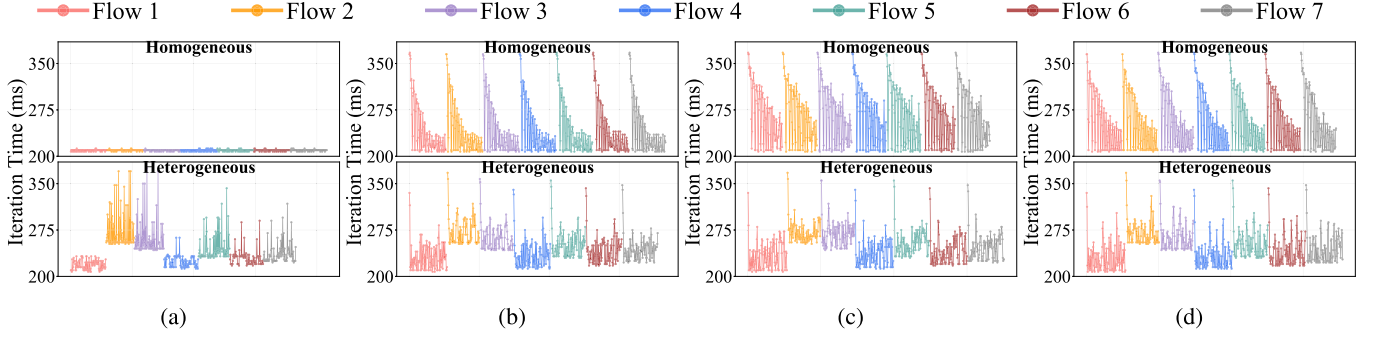


Fig. 11. Convergence performance for (a) Gsched, (b) SRTF, (c) MLTCP, and (d) DCTCP.

shows the largest fluctuations and requires the most iterations. Overall, the convergence order is Gsched > SRTF > DCTCP > MLTCP. These results show that Gsched stabilizes iteration time faster by coordinating computation and communication. This mainly comes from priority enforcement and work-conserving fallback, where high-priority flows leave the bottleneck earlier and blocked flows yield bandwidth to the next ready task.

Completely Heterogeneous Scenario: In a highly heterogeneous DNN training scenario, significant priority differences among tasks make scheduling behavior more complex in Fig. 11. Gsched, through a priority-aware mechanism, can provide more stable iteration times for high-priority tasks, while low-priority tasks yield in certain iteration rounds, introducing some delay and fluctuation, resulting in a controllable convergence process. In contrast, SRTF, MLTCP, and DCTCP exhibit similar performance in heterogeneous environments, with noticeable iteration time fluctuations across task flows, lacking sustained convergence, and showing less smooth and stable trends. This indicates that relying solely on flow scheduling is insufficient, highlighting the advantage of coordinated scheduling. More detailed results in Table III show that introducing a priority-aware scheduling strategy significantly improves the iteration time stability of high-priority tasks in heterogeneous scenarios.

G. Evaluation Scope

Our evaluation should be interpreted as RoCEv2 mechanism validation combined with a trace driven study. The physical testbed with RTX 4090 servers validates DSCP priority marking, SPQ enforcement, RoCEv2 backpressure, and the interaction between priority scheduling and congestion control. To study workload diversity beyond this testbed scale, we use traces derived from a dual server reference cluster. Therefore, the results validate the mechanism and its behavior under representative traces, but they do not replace a full production scale deployment study. We leave larger cluster validation with more GPU types and network topologies to future work.

VIII. CONCLUSION

Driven by the long-lived, bandwidth-intensive, and interdependent nature of DNN training traffic, Gsched coordinates flow control with priority-based scheduling to realize communication

interleaving on commodity switches. Its strict-priority enforcement mitigates bottleneck contention, while ATE-based ordering organizes service order according to task-level computation and communication profiles. More importantly, the local perturbation theorem provides a theoretical basis for decomposing global contention into per-bottleneck scheduling decisions with limited loss. Our evaluation shows that Gsched remains close to strong heuristics such as Crux in common settings and remains competitive under highly diverse workloads. We hope this work motivates further study of co-designed flow control, priority scheduling, and scalable decomposition for multi-tenant GPU clusters.

APPENDIX

Proof of Theorem 1

Proof: The dynamics of the N -flow system can be precisely mapped to a binary competition model, where the agents are the priority task J_p and the aggregated set of secondary-priority tasks J_{sec} . For simplicity, we define the throughput ratio as:

$$r = \frac{thr_{sec}}{thr_p} = \frac{C - thr_p}{thr_p} \quad (24)$$

According to the theorem condition $thr_p > C/2$, we have $0 < r < 1$. We further assume that in each iteration, the total logical data volume to be transmitted by J_p and J_{sec} is identical:

$$L_p = L_{sec} = L \quad (25)$$

In the exclusive transmission phases, the corresponding flow obtains the full link capacity C . We aim to prove the proposition $P(n)$: for all integers $n \geq 1$, in the n -th iteration, the data volumes transmitted in each phase satisfy:

$$\begin{aligned} L_1^{(n)} &= L \cdot (1 - r^{n-1}), & L_2^{(n)} &= L \cdot r^{n-1}, \\ L_3^{(n)} &= L \cdot r^n, & L_4^{(n)} &= L \cdot (1 - r^n). \end{aligned} \quad (26)$$

The corresponding phase durations are:

$$T_1^{(n)} = \frac{L}{C} \cdot (1 - r^{n-1}) \quad (27)$$

$$T_2^{(n)} = \frac{L}{thr_p} \cdot r^{n-1} \quad (28)$$

$$T_3^{(n)} = \frac{L}{C} \cdot (1 - r^n) \quad (29)$$

Verify that $P(1)$ holds. At the initial iteration $n = 1$, the system starts from a fully overlapping state, so the duration of phase 1 is zero, i.e., $L_1^{(1)} = 0$.

Verification of L formulas:

- For $L_1^{(1)}$: $L \cdot (1 - r^{1-1}) = L \cdot (1 - r^0) = 0$.
- For $L_2^{(1)}$: During phase 2, the priority task J_p transmits its entire data volume L , hence $L_2^{(1)} = L$, which matches the formula $L \cdot r^0 = L$.
- For $L_3^{(1)}$: While J_p transmits L during phase 2, J_{sec} concurrently transmits $L \cdot r$, so $L_3^{(1)} = L \cdot r$, consistent with $L \cdot r^1 = L \cdot r$.
- For $L_4^{(1)}$: After phase 2, the remaining data for J_{sec} is $L_4^{(1)} = L - L_3^{(1)} = L \cdot (1 - r)$, which matches $L \cdot (1 - r^1) = L \cdot (1 - r)$.

Verification of T formulas:

- For $T_1^{(1)}$: $\frac{L}{C} \cdot (1 - r^0) = 0$, consistent with phase 1 duration being zero.
- For $T_2^{(1)}$: $\frac{L}{thr_p} \cdot r^0 = \frac{L}{thr_p}$, representing the duration of phase 2 as the transmitted volume divided by the throughput of J_p .
- For $T_3^{(1)}$: $\frac{L}{C} \cdot (1 - r^1) = \frac{L}{C} \cdot (1 - r)$, the duration of phase 3 equals the remaining data volume of J_{sec} divided by its throughput.

Assume that for an arbitrary integer $k \geq 1$, the proposition $P(k)$ holds, i.e., the expressions for $L_1^{(k)}$, $L_2^{(k)}$, $L_3^{(k)}$, and $L_4^{(k)}$ are correct. We need to prove that $P(k+1)$ also holds given $P(k)$ is true. The iterative nature of the system implies that the state at iteration $k+1$ depends on the end state of iteration k . Deriving the L formulas for $k+1$:

$$L_1^{(k+1)} = L_4^{(k)} = L(1 - r^k) \quad (30)$$

$$L_2^{(k+1)} = L - L_1^{(k+1)} = L - L(1 - r^k) = L \cdot r^k \quad (31)$$

$$L_3^{(k+1)} = L_2^{(k+1)} \cdot r = L \cdot r^k \cdot r = L \cdot r^{k+1} \quad (32)$$

$$L_4^{(k+1)} = L - L_3^{(k+1)} = L - L \cdot r^{k+1} = L(1 - r^{k+1}) \quad (33)$$

All L formulas match the definitions in $P(k+1)$.

Deriving the T formulas for $k+1$:

$$T_1^{(k+1)} = \frac{L_1^{(k+1)}}{C} = \frac{L \cdot (1 - r^k)}{C} \quad (34)$$

$$T_2^{(k+1)} = \frac{L_2^{(k+1)}}{thr_p} = \frac{L \cdot r^k}{thr_p} \quad (35)$$

$$T_3^{(k+1)} = \frac{L_4^{(k+1)}}{C} = \frac{L \cdot (1 - r^{k+1})}{C} \quad (36)$$

All T formulas agree with the definitions in $P(k+1)$. Since all seven formulas hold for $k+1$ assuming they hold for k , the induction step is complete. By mathematical induction, the proposition $P(n)$ holds for all $n \geq 1$, proving the formulas for the durations of the three phases in iteration n : J_p exclusive,

overlap, and J_{sec} exclusive. Substituting r and letting L_p , L_{sec} be the loads of the priority and secondary tasks, the complete formulas are:

- J_p exclusive time:

$$T_1^{(n)} = \frac{L_p}{C} \cdot \left[1 - \left(\frac{C - thr_p}{thr_p} \right)^{n-1} \right] \quad (37)$$

- Overlapping communication time:

$$T_2^{(n)} = \frac{L_p}{thr_p} \cdot \left(\frac{C - thr_p}{thr_p} \right)^{n-1} \quad (38)$$

- J_{sec} exclusive time:

$$T_3^{(n)} = \frac{L_{sec}}{C} \cdot \left[1 - \left(\frac{C - thr_p}{thr_p} \right)^n \right] \quad (39)$$

■

REFERENCES

- [1] V. Addanki et al., "ABM: Active buffer management in datacenters," in *Proc. ACM SIGCOMM Conf.*, 2022, pp. 36–52.
- [2] V. Addanki et al., "{PowerTCP}: Pushing the performance limits of datacenter networks," in *Proc. 19th USENIX Symp. Netw. Syst. Des. Implementation*, 2022, pp. 51–70.
- [3] M. Alizadeh et al., "Hedera: Dynamic flow scheduling for data center networks," in *Proc. 7th USENIX Symp. Netw. Syst. Des. Implementation* 2010, pp. 89–92.
- [4] M. Alizadeh et al., "pFabric: Minimal near-optimal datacenter transport," in *Proc. ACM SIGCOMM Conf. SIGCOMM*, New York, NY, USA, 2013, pp. 435–446.
- [5] M. Alizadeh et al., "CONGA: Distributed congestion-aware load balancing for datacenters," in *Proc. ACM Conf. SIGCOMM*, 2014, pp. 503–514.
- [6] M. Alizadeh et al., "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM Conf.*, New York, NY, USA, 2010, pp. 63–74.
- [7] S. Arslan et al., "Bolt: {Sub-RTT} congestion control for {ultra-low} latency," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, 2023, pp. 219–236.
- [8] W. Bai et al., "Enabling {ECN} in {multi-service} {multi-queue} data centers," in *Proc. 13th USENIX Symp. Netw. Syst. Des. Implementation*, 2016, pp. 537–549.
- [9] W. Bai, L. Chen, K. Chen, D. Han, C. Tian, and H. Wang, "PIAS: Practical information-agnostic flow scheduling for commodity data centers," *IEEE/ACM Trans. Netw.*, vol. 25, no. 4, pp. 1954–1967, Aug. 2017.
- [10] A. Baiocchi et al., "Yeah-TCP: Yet another highspeed TCP," in *Proc. PFLDnet*, vol. 7, 2007, pp. 37–42.
- [11] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 8, pp. 1465–1480, Oct. 1995.
- [12] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [13] J. Cao et al., "Crux: GPU-efficient communication scheduling for deep learning training," in *Proc. ACM SIGCOMM Conf.*, New York, NY, USA, 2024, pp. 1–15.
- [14] Z. Chen et al., "{eTran}: Extensible kernel transport with {eBPF}," in *Proc. 22nd USENIX Symp. Netw. Syst. Des. Implementation*, 2025, pp. 407–425.
- [15] M. Chowdhury et al., "Efficient coflow scheduling with varys," *SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 443–454, Aug. 2014.
- [16] M. Chowdhury et al., "Efficient coflow scheduling without prior knowledge," *SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 393–406, Aug. 2015.
- [17] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [18] F. R. Dogar et al., "Decentralized task-aware scheduling for data center networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 431–442, 2014.
- [19] C. Du et al., "Cross-timeslot optimization for distributed GPU inference using reinforcement learning," 2025, *arXiv:2507.10259*.

- [20] Q. Duan, Z. Wang, Y. Xu, S. Liu, and J. Wu, "Mercury: A simple transport layer scheduler to accelerate distributed DNN training," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 350–359.
- [21] N. Dukkkipati et al., "Processor sharing flows in the Internet," in *Proc. 13th Int. Conf. Qual. Service*, Berlin, Heidelberg, Springer, 2005, pp. 271–285.
- [22] M. Gerla, M. Y. Sanadidi, Ren Wang, A. Zanella, C. Casetti, and S. Mascolo, "TCP Westwood: Congestion window control using bandwidth estimation," in *Proc. IEEE Glob. Telecommun. Conf.*, 2001, pp. 1698–1702.
- [23] J. Gu et al., "Tiresias: A GPU cluster manager for distributed deep learning," in *Proc. 16th USENIX Symp. Netw. Syst. Des. Implementation*, USENIX Association, Boston, MA, 2019, pp. 485–500.
- [24] S. Ha et al., "Cubic: A new TCP-friendly high-speed TCP variant," *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.
- [25] M. Handley et al., "Re-architecting datacenter networks and stacks for low latency and high performance," in *Proc. Conf. ACM Special Int. Group Data Commun.*, New York, NY, USA, 2017, pp. 29–42.
- [26] H. Sayed et al., "TicTac: Accelerating distributed deep learning with communication scheduling," in *Proc. Mach. Learn. Syst.*, 2019, pp. 418–430.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [28] K. He et al., "Presto: Edge-based load balancing for fast datacenter networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 465–478, 2015.
- [29] C.-Y. Hong et al., "Finishing flows quickly with preemptive scheduling," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 42, no. 4, pp. 127–138, 2012.
- [30] Q. Hu et al., "Characterization of large language model development in the datacenter," in *Proc. 21st USENIX Symp. Netw. Syst. Des. Implementation*, 2024, pp. 709–729.
- [31] V. Jacobson, "Congestion avoidance and control," in *Symp. Proc. Commun. Architectures Protoc.*, New York, NY, USA, 1988, pp. 314–329.
- [32] A. Jajoo et al., "Your coflow has many flows: Sampling them for fun and speed," in *Proc. USENIX Annu. Tech. Conf.*, Renton, WA, USA, 2019, pp. 833–848.
- [33] J. Suhas et al., "Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling," in *Proc. 29th Symp. Operating Syst. Princ.*, New York, NY, USA, 2023, pp. 642–657.
- [34] Y. Jiang et al., "A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, USENIX Association, 2020, pp. 463–479.
- [35] D. Katabi et al., "Congestion control for high bandwidth-delay product networks," *SIGCOMM Comput. Commun. Rev.*, vol. 32, no. 4, pp. 89–102, Aug. 2002.
- [36] G. Kumar et al., "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proc. Annu. Conf. ACM Special Int. Group Data Commun. Appl., Technol., Architectures, Protoc. Comput. Commun.*, 2020, pp. 514–528.
- [37] Y. Lei et al., "Fast: An efficient scheduler for all-to-all GPU communication," in *Proc. 23rd USENIX Symp. Netw. Syst. Des. Implementation (NSDI 26)*, 2026, pp. 2515–2531.
- [38] J. Li, S. Tripathi, L. Rastogi, Y. Lei, R. Pan, and Y. Xia, *IEEE Trans. Netw.*, vol. 34, pp. 2478–2497, 2026.
- [39] W. Li et al., "Understanding communication characteristics of distributed training," in *Proc. 8th Asia-Pacific Workshop Netw.*, 2024, pp. 1–8.
- [40] Y. Li et al., "HPCC: High precision congestion control," in *Proc. ACM Special Int. Group Data Commun.*, 2019, pp. 44–58.
- [41] Y. Liu et al., "RoBERTa: A robustly optimized bert pretraining approach," 2019, *arXiv:1907.11692*.
- [42] K. Mahajan et al., "Themis: Fair and efficient GPU cluster scheduling," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, 2020, pp. 289–304.
- [43] K. Mahajan et al., "Better Together: Jointly optimizing ML collective scheduling and execution planning using SYNDICATE," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, Boston, MA, USA, 2023, pp. 809–824.
- [44] L. Mai et al., "Optimizing network performance in distributed machine learning," on *Proc. 7th USENIX Workshop Hot Topics Cloud Comput.*, USENIX Association, Santa Clara, CA, USA, 2015.
- [45] L. Martin, "CamemBERT: A tasty french language model," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 7203–7219.
- [46] W. M. Mellette et al., "Expanding across time to deliver bandwidth efficiency and low latency," in *Proc. 17th USENIX Symp. Netw. Syst. Des. Implementation*, 2020, pp. 1–18.
- [47] R. Mittal et al., "Timely: RTT-based congestion control for the datacenter," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 537–550, 2015.
- [48] J. Mohan et al., "Looking beyond {GPUs} for {DNN} scheduling on {multi-tenant} clusters," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 579–596.
- [49] B. Montazeri et al., "Homa: A receiver-driven low-latency transport protocol using network priorities," in *Proc. Conf. ACM Special Int. Group Data Commun.*, New York, NY, USA, 2018, pp. 221–235.
- [50] R. Pan et al., "Efficient flow scheduling in distributed deep learning training with echelon formation," in *Proc. 21st ACM Workshop Hot Topics Netw.*, New York, NY, USA, 2022, pp. 93–100.
- [51] Y. Peng et al., "Optimus: An efficient dynamic resource scheduler for deep learning clusters," in *Proc. 13th EuroSys Conf.*, New York, NY, USA, 2018, pp. 1–14.
- [52] Y. Peng et al., "A generic communication scheduler for distributed dnn training acceleration," in *Proc. 27th ACM Symp. Operating Syst. Princ.*, New York, NY, USA, 2019, pp. 16–29.
- [53] J. Perry et al., "Fastpass: A centralized "zero-queue" datacenter network," in *Proc. ACM Conf. SIGCOMM*, New York, NY, USA, 2014, pp. 307–318.
- [54] A. Qiao et al., "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *Proc. 15th USENIX Symp. Operating Syst. Des. Implementation*, 2021, pp. 1–18.
- [55] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019, Art. no. 9.
- [56] S. Rajasekaran et al., "CASSINI: Network-aware job scheduling in machine learning clusters," in *Proc. 21st USENIX Symp. Networked Syst. Des. Implementation*, Santa Clara, CA, USA, 2024, pp. 1403–1420.
- [57] S. Rajasekaran et al., "MLTCP: A distributed technique to approximate centralized flow scheduling for machine learning," in *Proc. 23rd ACM Workshop Hot Topics Netw.*, 2024, pp. 167–176.
- [58] A. Roy et al., "Inside the social network's (datacenter) network," in *Proc. ACM Conf. Special Int. Group Data Commun.*, 2015, pp. 123–137.
- [59] L. Schrage, "A proof of the optimality of the shortest remaining processing time discipline," *Operations Res.*, vol. 16, no. 3, pp. 687–690, 1968.
- [60] A. Shah et al., "TACCL: Guiding collective algorithm synthesis using communication sketches," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, USENIX Association, Boston, MA, USA, 2023, pp. 593–612.
- [61] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [62] C. Hwan Song et al., "Network load balancing with in-network reordering support for RDMA," in *Proc. ACM SIGCOMM Conf.*, 2023, pp. 816–831.
- [63] A. Sultana et al., "Hadar: Heterogeneity-aware optimization-based online scheduling for deep learning cluster," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2024, pp. 681–691.
- [64] P. Taheri et al., "RoCC: Robust congestion control for RDMA," in *Proc. 16th Int. Conf. Emerg. Netw. Experiments Technol.*, New York, NY, USA, 2020, pp. 17–30.
- [65] Y. Gao, B. Hu, M. B. Mashhadi, A. -L. Jin, P. Xiao, and C. Wu, "US-Byte: An efficient communication framework for scheduling unequal-sized tensor blocks in distributed deep learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 1, pp. 123–139, Jan. 2024.
- [66] Y. Gao, B. Hu, M. B. Mashhadi, W. Wang, R. Tafazolli, and M. Debbah, "A dynamic sliding window based tensor communication scheduling framework for distributed deep learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 2, pp. 1080–1095, Mar./Apr. 2025.
- [67] R. Pan et al., "Efficient flow scheduling in distributed deep learning training with echelon formation," in *Proc. 21st ACM Workshop Hot Topics Netw.*, 2022, pp. 93–100.
- [68] B. Vamanan et al., "Deadline-aware datacenter TCP (D2TCP)," in *Proc. ACM SIGCOMM Conf. Appl., Technol., Architectures, Protoc. Comput. Commun.*, New York, NY, USA, 2012, pp. 115–126.
- [69] E. Vanini et al., "Let it flow: Resilient asymmetric load balancing with flowlet switching," in *Proc. 14th USENIX Symp. Netw. Syst. Des. Implementation*, 2017, pp. 407–420.
- [70] W. Xiao et al., "Gandiva: Introspective cluster scheduling for deep learning," in *Proc. 13th USENIX Symp. Operating Syst. Des. Implementation*, Carlsbad, CA, USA, 2018, pp. 595–610.
- [71] W. Xiao et al., "{AntMan}: Dynamic scaling on {GPU} clusters for deep learning," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 533–548.
- [72] H. Xu et al., "RailS: Load balancing for all-to-all communication in distributed mixture-of-experts training," *IEEE Trans. Netw.*, vol. 34, pp. 4431–4448, 2026.

- [73] L. Xu, K. Harfoush, and I. Rhee, "Binary increase congestion control (BIC) for fast long-distance networks," in *Proc. IEEE INFOCOM*, 2004, pp. 2514–2524.
- [74] S. Yan et al., "ACC: Automatic ECN tuning for high-speed datacenter networks," in *Proc. ACM SIGCOMM Conf.*, 2021, pp. 384–397.
- [75] J. Zhang et al., "Micro-burst aware ECN in multi-queue data centers: Algorithm and implementation," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2384–2398, May/Jun. 2024.
- [76] Q. Zhang, "High-resolution measurement of data center microbursts," in *Proc. Internet Meas. Conf.*, 2017, pp. 78–85.
- [77] Y. Zhao et al., "Multi-resource interleaving for deep learning training," in *Proc. ACM SIGCOMM Conf.*, New York, NY, USA, 2022, pp. 428–440.
- [78] P. Zheng et al., "Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, Boston, MA, USA, 2023, pp. 703–723.
- [79] Y. Zheng, K. Zhang, H. Qi, Z. Shu, and Q. Zhao, "Model predictive congestion control for data center networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 3070–3086, 2026.
- [80] Y. Zhou et al., "UCCL-Tran: An extensible software transport layer for GPU networking," in *Proc. USENIX OSDI*, 2026.
- [81] Y. Zhu et al., "Congestion control for large-scale RDMA deployments," in *Proc. ACM Conf. Special Int. Group Data Commun.*, New York, NY, USA, 2015, pp. 523–536.



Heng Xu received the bachelor's and master's degrees from Beijing Jiaotong University, Beijing, China, in 2022 and 2025, respectively. His research interests include network transport and scheduling, edge computing, and data center systems.



Chengze Du received the undergraduate degree from Beijing University of Posts and Telecommunications, Beijing, China. His research interests include the intersection of AI, security, and networking. His work includes network tomography, topology obfuscation, membership inference defenses.



Zhiwei Yu received the bachelor's degree from Beijing University of Posts and Telecommunications, Beijing, China. He is currently working toward the master's degree with Tsinghua University, Beijing. His research interests include data center networks and large language models.



Letian Li received the B.S. degree in communications engineering and the Ph.D. degree in information and communications engineering from Beijing Jiaotong University, Beijing, China, in 2018 and 2023, respectively. He is currently a Research Assistant with the Department of Information Engineering, Chinese University of Hong Kong, Shatin, Hong Kong. His research interests include real-time interaction and transmission protocols.



Ying Zhou currently working toward the doctoral degree in information and communication engineering with the School of Electronic Information Engineering, Beijing Jiaotong University, Beijing, China, in 2022. His research interests include network optimization, artificial intelligence, and image processing.



Bo Liu received the B.Eng. and M.Eng. degrees from the Wuhan University of Technology, Wuhan, China, in 2014 and 2017, respectively, and the PhD degree in electrical and electronic engineering from the University of Manchester, Manchester, U.K., in 2021. He is currently an Assistant Professor with Shenzhen University of Advanced Technology, Shenzhen, China. His research interests include distributed optimization, distributed machine learning, and their applications.



Jialong Li (Member, IEEE) received the B.E. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2016 and 2021, respectively. He was a Postdoctoral Researcher with the Max Planck Institute for Informatics (MPI-INF). He is currently an Assistant Professor with Shenzhen University of Advanced Technology, Shenzhen, China. His work has been published in top-tier venues such as SIGCOMM, NSDI, and IEEE TRANSACTIONS ON NETWORKING (ToN).